

Style_Transfer_Exercise...
File

take some time to understand this paper I have an assignment based on this paper

Image Style Transfer Using Convolutional Neural Networks

Leon A. Gatys

Centre for Integrative Neuroscience, University of Tübingen, Germany

Bernstein Center for Computational Neuroscience, Tübingen, Germany

Graduate School of Neural Information Processing, University of Tübingen, Germany

leon.gatys@bethgelab.org

Alexander S. Ecker

Centre for Integrative Neuroscience, University of Tübingen, Germany

Bernstein Center for Computational Neuroscience, Tübingen, Germany

Max Planck Institute for Biological Cybernetics, Tübingen, Germany

Baylor College of Medicine, Houston, TX, USA

Matthias Bethge

Centre for Integrative Neuroscience, University of Tübingen, Germany

Bernstein Center for Computational Neuroscience, Tübingen, Germany

Max Planck Institute for Biological Cybernetics, Tübingen, Germany

Abstract

Rendering the semantic content of an image in different styles is a difficult image processing task. Arguably, a major limiting factor for previous approaches has been the lack of image representations that explicitly represent semantic information and, thus, allow to separate image content from style. Here we use image representations derived from Convolutional Neural Networks optimised for object recognition, which make high level image information explicit. We introduce A Neural Algorithm of Artistic Style that can separate and recombine the image content and style of natural images. The algorithm allows us to produce new images of high perceptual quality that combine the content of an arbitrary photograph with the appearance of numerous well-known artworks. Our results provide new insights into the deep image representations learned by Convolutional Neural Networks and demonstrate their potential for high level image synthesis and manipulation.

1. Introduction

Transferring the style from one image onto another can be considered a problem of texture transfer. In texture transfer the goal is to synthesise a texture from a source image while constraining the texture synthesis in order to preserve the semantic content of a target image. For texture synthesis there exist a large range of powerful non-parametric algorithms that can synthesise photorealistic natural textures by resampling the pixels of a given source texture [7, 30, 8, 20]. Most previous texture transfer algorithms rely on these non-parametric methods for texture synthesis while using different ways to preserve the structure of the target image. For

instance, Efros and Freeman introduce a correspondence map that includes features of the target image such as image intensity to constrain the texture synthesis procedure [8]. Hertzman et al. use image analogies to transfer the texture from an already stylised image onto a target image [13]. Ashikhmin focuses on transferring the high-frequency texture information while preserving the coarse scale of the target image [1]. Lee et al. improve this algorithm by additionally informing the texture transfer with edge orientation information [22].

Although these algorithms achieve remarkable results, they all suffer from the same fundamental limitation: they use only low-level image features of the target image to inform the texture transfer. Ideally, however, a style transfer algorithm should be able to extract the semantic image content from the target image (e.g. the objects and the general scenery) and then inform a texture transfer procedure to render the semantic content of the target image in the style of the source image. Therefore, a fundamental prerequisite is to find image representations that independently model variations in the semantic image content and the style in which

1 2414

b

a e

c

d

Style Reconstructions

Style

Representations

Input image

Content

Representations

Convolutional Neural Network

b

a e

c

d

Content Reconstructions

Figure 1. Image representations in a Convolutional Neural Network (CNN). A given input image is represented as a set of filtered images at each processing stage in the CNN. While the number of different filters increases along the processing hierarchy, the size of the filtered

images is reduced by some downsampling mechanism (e.g. max-pooling) leading to a decrease in the total number of units per layer of the

network. Content Reconstructions. We can visualise the information at different processing stages in the CNN by reconstructing the input image from only knowing the network's responses in a particular layer. We reconstruct the input image from from layers 'conv1 2' (a), 'conv2 2' (b), 'conv3 2' (c), 'conv4 2' (d) and 'conv5 2' (e) of the original VGG-Network. We find that reconstruction from lower layers is

almost perfect (a–c). In higher layers of the network, detailed pixel information is lost while the high-level content of the image is preserved

(d,e). Style Reconstructions. On top of the original CNN activations we use a feature space that captures the texture information of an input image. The style representation computes correlations between the different features in different layers of the CNN. We reconstruct

the style of the input image from a style representation built on different subsets of CNN layers ('conv1 1' (a), 'conv1 1' and 'conv2 1' (b), 'conv1 1', 'conv2 1' and 'conv3 1' (c), 'conv1 1', 'conv2 1', 'conv3 1' and 'conv4 1' (d), 'conv1 1', 'conv2 1', 'conv3 1', 'conv4 1' and 'conv5 1' (e). This creates images that match the style of a given image on an increasing scale while discarding information of the global arrangement of the scene.

it is presented. Such factorised representations were pre-trained with sufficient labeled data on specific tasks such as object recognition learn to extract high-level image content in generic feature representations that generalise across and characters in different font styles [29] or handwritten datasets [6] and even to other visual information processing digits and house numbers [17].

tasks [19, 4, 2, 9, 23], including texture recognition [5] and To generally separate content from style in natural im-artistic style classification [15].

ages is still an extremely difficult problem. However, the re-In this work we show how the generic feature representation advance of Deep Convolutional Neural Networks [18] tations learned by high-performing Convolutional Neural has produced powerful computer vision systems that learn Networks can be used to independently process and ma-to extract high-level semantic information from natural im-nipulate the content and the style of natural images. We ages. It was shown that Convolutional Neural Networks introduce A Neural Algorithm of Artistic Style, a new algo-2415

rithm to perform image style transfer. Conceptually, it is a texture transfer algorithm that constrains a texture synthesis method by feature representations from state-of-the-art Convolutional Neural Networks. Since the texture model is also based on deep image representations, the style transfer method elegantly reduces to an optimisation problem within a single neural network. New images are generated by performing a pre-image search to match feature representations of example images. This general approach has been used before in the context of texture synthesis [12, 25, 10] and to improve the understanding of deep image representations [27, 24]. In fact, our style transfer algorithm combines a parametric texture model based on Convolutional Neural Networks [10] with a method to invert their image representations [24].

2. Deep image representations

The results presented below were generated on the basis of the VGG network [28], which was trained to perform object recognition and localisation [26] and is described extensively in the original work [28]. We used the feature space provided by a normalised version of the 16 convolutional and 5 pooling layers of the 19-layer VGG network. We normalized the network by scaling the weights such that the mean activation of each convolutional filter over images and positions is equal to one. Such re-scaling can be done for the VGG network without changing its output, because it contains only rectifying linear activation functions and no normalization or pooling over feature maps. We do not use any of the fully connected layers. The model is publicly available and can be explored in the caffe-framework [14].

For image synthesis we found that replacing the maximum pooling operation by average pooling yields slightly more appealing results, which is why the images shown were generated with average pooling.

2.1. Content representation

Generally each layer in the network defines a non-linear filter bank whose complexity increases with the position of the layer in the network. Hence a given input image I is encoded in each layer of the Convolutional Neural Network by the filter responses to that image. A layer with N_l distinct filters has N_l feature maps each of size M_l , where M_l is the height times the width of the feature map. So the responses in a layer l can be stored in a matrix $F_l \in \mathbb{R}^{N_l \times M_l}$ where F_l

f_{ij} is the activation of the i th filter at position j in layer l .

To visualise the image information that is encoded at different layers of the hierarchy one can perform gradient descent on a white noise image to find another image that matches the feature responses of the original image (Fig 1, content reconstructions) [24]. Let I and $\tilde{I}$ be the original image and the image that is generated, and F_l and $\tilde{F}_l$ their respective feature representation in layer l . We then define the squared-error loss between the two feature representations

$$L_{\text{content}}(I, \tilde{I}) = \frac{1}{2} \sum_{l=1}^L \sum_{i,j} (f_{ij} - \tilde{f}_{ij})^2 \quad (1)$$

$$\frac{\partial L_{\text{content}}}{\partial f_{ij}} = f_{ij} - \tilde{f}_{ij}$$

The derivative of this loss with respect to the activations in layer l equals

$$\frac{\partial L_{\text{content}}}{\partial f_{ij}} = \begin{cases} f_{ij} - \tilde{f}_{ij} & \text{if } f_{ij} > 0 \\ 0 & \text{if } f_{ij} = 0 \\ f_{ij} - \tilde{f}_{ij} & \text{if } f_{ij} < 0 \end{cases} \quad (2)$$

from which the gradient with respect to the image $\tilde{I}$ can be computed using standard error back-propagation (Fig 2, right). Thus we can change the initially random image $\tilde{I}$ until it generates the same response in a certain layer of the Convolutional Neural Network as the original image I .

When Convolutional Neural Networks are trained on object recognition, they develop a representation of the image that makes object information increasingly explicit along the processing hierarchy [10]. Therefore, along the processing hierarchy of the network, the input image is transformed into representations that are increasingly sensitive to the actual content of the image, but become relatively invariant to its precise appearance. Thus, higher layers in the network capture the high-level content in terms of objects and their arrangement in the input image but do not constrain the ex-

act pixel values of the reconstruction very much (Fig 1, content reconstructions d, e). In contrast, reconstructions from the lower layers simply reproduce the exact pixel values of the original image (Fig 1, content reconstructions a–c). We therefore refer to the feature responses in higher layers of the network as the content representation.

2.2. Style representation

To obtain a representation of the style of an input image, we use a feature space designed to capture texture information [10]. This feature space can be built on top of the filter responses in any layer of the network. It consists of the correlations between the different filter responses, where the expectation is taken over the spatial extent of the feature maps. These feature correlations are given by the Gram matrix $G^l \in \mathbb{R}^{N_l \times N_l}$, where G^l_{ij} is the inner product between the vectorised feature maps i and j in layer l :

$$G^l_{ij} = \frac{1}{K} \sum_k F^l_{ik} F^l_{jk} \quad (3)$$

By including the feature correlations of multiple layers, we obtain a stationary, multi-scale representation of the input image, which captures its texture information but not the global arrangement. Again, we can visualise the information captured by these style feature spaces built on different

2416
512
1
...
512
...
conv5_
4
3 2
1
conv5_ 4
3 2
1
pool4
pool4
1
256
...
conv4_
4
3 2
1
conv4_ 4
3 2
1
pool3
pool3
1
128
...
conv3_
4

```

3 2
1
conv3_4
3 2
1
pool2
pool2
64
...
1
1
# feature
maps
conv2_
1
2
conv2_ 2
1
pool1
pool1
conv1_ 2
1
conv1_ 2
1
input Gradient
descent
input

```

Figure 2. Style transfer algorithm. First content and style features are extracted and stored. The style image  is passed through the network and its style representation A_I on all layers included are computed and stored (left). The content image  is passed through the network and the content representation P_I in one layer is stored (right). Then a random white noise image  is passed through the network and its style features G_I and content features F_I are computed. On each layer included in the style representation, the element-wise mean squared difference between G_I and A_I is computed to give the style loss L_{style} (left). Also the mean squared difference between F_I and P_I is computed to give the content loss $L_{content}$ (right). The total loss L_{total} is then a linear combination between the content and the style loss.

Its derivative with respect to the pixel values can be computed using error back-propagation (middle). This gradient is used to iteratively update the image  until it simultaneously matches the style features of the style image  and the content features of the content image  (middle, bottom).

layers of the network by constructing an image that matches the style representation of a given input image (Fig 1, style reconstructions). This is done by using gradient descent from a white noise image to minimise the mean-squared distance between the entries of the Gram matrices from the original image and the Gram matrices of the image to be generated [10, 25].

Let  and  be the original image and the image that is generated, and A_I and G_I their respective style representation in layer I . The contribution of layer I to the total loss is then

$E_I =$
1

$$\begin{aligned} & 4N^2 \\ & \sum_{i,j} \\ & |I_{i,j} - A_{i,j}|^2 \\ & \text{and the total style loss is} \\ & L_{\text{style}} = \sum_{l=1}^L w_l E_l \end{aligned} \tag{5}$$

where w_l are weighting factors of the contribution of each layer to the total loss (see below for specific values of w_l in our results). The derivative of E_l with respect to the activations in layer l can be computed analytically:

$$\frac{\partial E_l}{\partial F^l_{i,j}} = \begin{cases} 1 & \text{if } F^l_{i,j} > 0 \\ 0 & \text{if } F^l_{i,j} = 0 \\ -1 & \text{if } F^l_{i,j} < 0 \end{cases} \tag{6}$$

The gradients of E_l with respect to the pixel values can be readily computed using standard error back-propagation (Fig 2, left).

2.3. Style transfer

To transfer the style of an artwork onto a photograph we synthesise a new image that simultaneously matches the content representation of the photograph and the style representation of the artwork (Fig 2). Thus we jointly minimise the distance of the feature representations of a white noise image from the content representation of the photograph in one layer and the style representation of the painting defined on a number of layers of the Convolutional Neural Network. The loss function we minimise is

- A
- B
- C
- D
- E
- F

Figure 3. Images that combine the content of a photograph with the style of several well-known artworks. The images were created by finding an image that simultaneously matches the content representation of the photograph and the style representation of the artwork.

The original photograph depicting the Neckarfront in Tübingen, Germany, is shown in A (Photo: Andreas Praefcke). The painting that provided the style for the respective generated image is shown in the bottom left corner of each panel. B The Shipwreck of the Minotaur

by J.M.W. Turner, 1805. C The Starry Night by Vincent van Gogh, 1889. D Der Schrei by Edvard Munch, 1893. E Femme nue assise by Pablo Picasso, 1910. F Composition VII by Wassily Kandinsky, 1913.

2418

10-4 10-3

$$L_{\text{total}}(\mathbf{x}) = \alpha L_{\text{content}}(\mathbf{x}) + \beta L_{\text{style}}(\mathbf{x}) \quad (7)$$

where α and β are the weighting factors for content and style reconstruction, respectively. The gradient with respect to the pixel values $\partial L_{\text{total}}$

$\partial \mathbf{x}$ can be used as input for some nu-

merical optimisation strategy. Here we use L-BFGS [32], which we found to work best for image synthesis. To extract image information on comparable scales, we always resized the style image to the same size as the content image before computing its feature representations. Finally, note that in difference to [24] we do not regularise our synthesis results with image priors. It could be argued, though, that the texture features from lower layers in the network act as a specific image prior for the style image. Additionally some differences in the image synthesis are expected due to the different network architecture and optimisation algorithm we use.

3. Results

The key finding of this paper is that the representations of content and style in the Convolutional Neural Network are well separable. That is, we can manipulate both representations independently to produce new, perceptually meaningful images. To demonstrate this finding, we generate images that mix the content and style representation from two different source images. In particular, we match the content representation of a photograph depicting the riverfront of the Neckar river in Tübingen, Germany and the style representations of several well-known artworks taken from different periods of art (Fig 3). The images shown in Fig 3 were synthesised by matching the content representation on layer 'conv4_2' and the style representation on layers 'conv1_1', 'conv2_1', 'conv3_1', 'conv4_1' and 'conv5_1' ($w_l = 1/5$ in those layers, $w_l = 0$ in all other layers). The ratio α/β was either 1×10^{-3} (Fig 3 B), 8×10^{-4} (Fig 3 C), 5×10^{-3} (Fig 3 D), or 5×10^{-4} (Fig 3 E, F).

3.1. Trade-off between content and style matching

Of course, image content and style cannot be completely disentangled. When synthesising an image that combines the content of one image with the style of another, there usually does not exist an image that perfectly matches both constraints at the same time. However, since the loss function we minimise during image synthesis is a linear combination between the loss functions for content and style respectively, we can smoothly regulate the emphasis on either reconstructing the content or the style (Fig 4). A strong emphasis on style will result in images that match the appearance of the artwork, effectively giving a texturised version of it, but show hardly any of the photograph's content ($\alpha/\beta = 1 \times 10^{-4}$, Fig 4, top left). When placing strong

10-2

10-1

Figure 4. Relative weighting of matching content and style of the respective source images. The ratio α/β between matching the content and matching the style increases from top left to bottom right. A high emphasis on the style effectively produces a tex-

turised version of the style image (top left). A high emphasis on the content produces an image with only little stylisation (bottom right). In practice one can smoothly interpolate between the two extremes.

emphasis on content, one can clearly identify the photograph, but the style of the painting is not as well-matched ($\alpha/\beta = 1 \times 10^{-1}$, Fig 4, bottom right). For a specific pair of content and style images one can adjust the trade-off between content and style to create visually appealing images.

3.2. Effect of different layers of the Convolutional Neural Network

Another important factor in the image synthesis process is the choice of layers to match the content and style representation on. As outlined above, the style representation is a multi-scale representation that includes multiple layers of the neural network. The number and position of these layers determines the local scale on which the style is matched, leading to different visual experiences (Fig 1, style reconstructions). We find that matching the style representations up to higher layers in the network preserves local image structures an increasingly large scale, leading to a smoother and more continuous visual experience. Thus, the visually most appealing images are usually created by matching the style representation up to high layers in the network, which is why for all images shown we match the style features in layers 'conv1 1', 'conv2 1', 'conv3 1', 'conv4 1' and 'conv5 1' of the network.

To analyse the effect of using different layers to match the content features, we present a style transfer result obtained by stylising a photograph with the same artwork and parameter configuration ($\alpha/\beta = 1 \times 10^{-3}$), but in one

2419
Content Image

A B

C

Conv2_2

Conv4_2

Figure 6. Initialisation of the gradient descent. A Initialised from the content image. B Initialised from the style image. C Four samples of images initialised from different white noise images. For all images the ratio α/β was equal to 1×10^{-3}

Figure 5. The effect of matching the content representation in different layers of the network. Matching the content on layer 'conv2 2' preserves much of the fine structure of the original photograph and the synthesised image looks as if the texture of the painting is simply blended over the photograph (middle). When matching the content on layer 'conv4 2' the texture of the painting and the content of the photograph merge together such that the content of the photograph is displayed in the style of the painting (bottom). Both images were generated with the same choice of parameters ($\alpha/\beta = 1 \times 10^{-3}$). The painting that served as the style image is shown in the bottom left corner and is named Jesuiten III by Lyonel Feininger, 1915.

3.3. Initialisation of gradient descent

We have initialised all images shown so far with white noise. However, one could also initialise the image synthesis with either the content image or the style image. We explored these two alternatives (Fig 6 A, B): although they bias the final image somewhat towards the spatial structure of the initialisation, the different initialisations do not seem

to have a strong effect on the outcome of the synthesis procedure. It should be noted that only initialising with noise allows to generate an arbitrary number of new images (Fig 6 C). Initialising with a fixed image always deterministically leads to the same outcome (up to stochasticity in the gradient descent procedure).

matching the content features on layer 'conv2_2' and in the other on layer 'conv4_2' (Fig 5). When matching the content on a lower layer of the network, the algorithm matches much of the detailed pixel information in the photograph and the generated image appears as if the texture of the artwork is merely blended over the photograph (Fig 5, middle). In contrast, when matching the content features on a higher layer of the network, detailed pixel information of the photograph is not as strongly constraint and the texture of the artwork and the content of the photograph are properly merged. That is, the fine structure of the image, for example the edges and colour map, is altered such that it agrees with the style of the artwork while displaying the content of the photograph (Fig 5, bottom).

3.4. Photorealistic style transfer

Thus far the focus of this paper was on artistic style transfer. In general though, the algorithm can transfer the style between arbitrary images. As an example we transfer the style of a photograph of New York by night onto an image of London in daytime (Fig 7). Although photorealism is not fully preserved, the synthesised image resembles much of the colours and lightning of the style image and to some extent displays an image of London by night.

2420

4. Discussion

Here we demonstrate how to use feature representations from high-performing Convolutional Neural Networks to transfer image style between arbitrary images. While we are able to show results of high perceptual quality, there are still some technical limitations to the algorithm.

Probably the most limiting factor is the resolution of the synthesised images. Both, the dimensionality of the optimisation problem as well as the number of units in the Convolutional Neural Network grow linearly with the number of pixels. Therefore the speed of the synthesis procedure depends heavily on image resolution. The images presented in this paper were synthesised in a resolution of about 512×512 pixels and the synthesis procedure could take up to an hour on a Nvidia K40 GPU (depending on the exact image size and the stopping criteria for the gradient descent). While this performance currently prohibits online and interactive applications of our style transfer algorithm, it is likely that future improvements in deep learning will also increase the performance of this method.

Another issue is that synthesised images are sometimes subject to some low-level noise. While this is less of an issue in the artistic style transfer, the problem becomes more apparent when both, content and style images, are photographs and the photorealism of the synthesised image is affected. However, the noise is very characteristic and appears to resemble the filters of units in the network. Thus it could be possible to construct efficient de-noising techniques to post-process the images after the optimisation procedure.

Artistic stylisation of images is traditionally studied in computer graphics under the label of non-photorealistic rendering. Apart from the work on texture transfer, common approaches are conceptually quite different to our work in that they give specialised algorithms to render a source image in one specific style. For a recent review of the field we refer the reader to [21].

The separation of image content from style is not necessarily a well defined problem. This is mostly because it is not clear what exactly defines the style of an image. It might be the brush strokes in a painting, the colour map, certain dominant forms and shapes, but also the composition of a scene and the choice of the subject of the image – and probably it is a mixture of all of them and many more. Therefore, it is generally not clear if image content and style can be completely separated at all – and if so, how. For example it is not possible to render an image in the style of van Gogh's "Starry Night" without having image structures that resemble the stars. In our work we consider style transfer to be successful if the generated image 'looks like' the style image but shows the objects and scenery of the content image. We are fully aware though that this evaluation criterion is neither mathematically precise nor universally

Style Image Content Image
Figure 7. Photorealistic style transfer. The style is transferred from a photograph showing New York by night onto a picture showing London by day. The image synthesis was initialised from the content image and the ratio α/β was equal to 1×10^2 agreed upon.

Nevertheless, we find it truly fascinating that a neural system, which is trained to perform one of the core computational tasks of biological vision, automatically learns image representations that allow – at least to some extent – the separation of image content from style. One explanation may be that when learning object recognition, the network has to become invariant to all image variation that preserves object identity. Representations that factorise the variation in the content of an image and the variation in its appearance would be extremely practical for this task. In light of the striking similarities between performance-optimised artificial neural networks and biological vision [11, 31, 3, 19, 16], it is thus tempting to speculate that the human ability to abstract content from style – and therefore our ability to create and enjoy art – might also be primarily a preeminent signature of the powerful inference capabilities of our visual system.

References

- [1] N. Ashikhmin. Fast texture transfer. IEEE Computer Graphics and Applications, 23(4):38–43, July 2003. 1
- [2] M. Berning, K. M. Boergens, and M. Helmstaedter. SegEM: Efficient Image Analysis for High-Resolution Connections. Neuron, 87(6):1193–1206, Sept. 2015. 2
- [3] C. F. Cadieu, H. Hong, D. L. K. Yamins, N. Pinto, D. Ardila, E. A. Solomon, N. J. Majaj, and J. J. DiCarlo. Deep Neural Networks Rival the Representation of Primate IT Cortex for Core Visual Object Recognition. PLoS Comput Biol, 10(12):e1003963, Dec. 2014. 8
- [4] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille. Semantic Image Segmentation with

Deep Convolutional Nets and Fully Connected CRFs.

arXiv:1412.7062 [cs], Dec. 2014. arXiv: 1412.7062. 2

[5] M. Cimpoi, S. Maji, and A. Vedaldi. Deep filter banks for texture recognition and segmentation. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pages 3828–3836, 2015. 2

[6] J. Donahue, Y. Jia, O. Vinyals, J. Hoffman, N. Zhang, E. Tzeng, and T. Darrell. DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition. arXiv:1310.1531 [cs], Oct. 2013. arXiv: 1310.1531. 2

[7] A. Efros and T. K. Leung. Texture synthesis by non-parametric sampling. In Computer Vision, 1999. The Proceedings of the Seventh IEEE International Conference on, volume 2, pages 1033–1038. IEEE, 1999. 1

[8] A. A. Efros and W. T. Freeman. Image quilting for texture synthesis and transfer. In Proceedings of the 28th annual conference on Computer graphics and interactive techniques, pages 341–346. ACM, 2001. 1

[9] D. Eigen and R. Fergus. Predicting Depth, Surface Normals and Semantic Labels With a Common Multi-Scale Convolutional Architecture. pages 2650–2658, 2015. 2

[10] L. A. Gatys, A. S. Ecker, and M. Bethge. Texture Synthesis Using Convolutional Neural Networks. In Advances in Neural Information Processing Systems 28, 2015. 3, 4

[11] U. G. and M. A. J. v. Gerven. Deep Neural Networks Reveal a Gradient in the Complexity of Neural Representations across the Ventral Stream. The Journal of Neuroscience, 35(27):10005–10014, July 2015. 8

[12] D. J. Heeger and J. R. Bergen. Pyramid-based Texture Analysis/Synthesis. In Proceedings of the 22Nd Annual Conference on Computer Graphics and Interactive Techniques, SIGGRAPH '95, pages 229–238, New York, NY, USA, 1995. ACM. 3

[13] A. Hertzmann, C. E. Jacobs, N. Oliver, B. Curless, and D. H. Salesin. Image analogies. In Proceedings of the 28th annual conference on Computer graphics and interactive techniques, pages 327–340. ACM, 2001. 1

[14] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell. Caffe: Convolutional architecture for fast feature embedding. In Proceedings of the ACM International Conference on Multimedia, pages 675–678. ACM, 2014. 3

[15] S. Karayev, M. Trentacoste, H. Han, A. Agarwala, T. Darrell, A. Hertzmann, and H. Winnemoeller. Recognizing image style. arXiv preprint arXiv:1311.3715, 2013. 2

[16] S.-M. Khaligh-Razavi and N. Kriegeskorte. Deep Supervised, but Not Unsupervised, Models May Explain IT Cortical Representation. PLoS Comput Biol, 10(11):e1003915, Nov. 2014. 8

[17] D. P. Kingma, S. Mohamed, D. Jimenez Rezende, and M. Welling. Semi-supervised Learning with Deep Generative Models. In Z. Ghahramani, M. Welling, C. Cortes, N. D. Lawrence, and K. Q. Weinberger, editors, Advances in Neural Information Processing Systems 27, pages 3581–3589. Curran Associates, Inc., 2014. 2

[18] A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. In Advances in neural information processing systems, pages 1097–1105, 2012. 2

[19] M. Kümmerer, L. Theis, and M. Bethge. Deep Gaze I: Boost-

- ing Saliency Prediction with Feature Maps Trained on ImageNet. In ICLR Workshop, 2015. 2, 8
- [20] V. Kwatra, A. Schödl, I. Essa, G. Turk, and A. Bobick. Graphcut textures: image and video synthesis using graph cuts. In ACM Transactions on Graphics (ToG), volume 22, pages 277–286. ACM, 2003. 1
- [21] J. E. Kyprianidis, J. Collomosse, T. Wang, and T. Isenberg. State of the “Art”: A Taxonomy of Artistic Stylization Techniques for Images and Video. Visualization and Computer Graphics, IEEE Transactions on, 19(5):866–885, 2013. 8
- [22] H. Lee, S. Seo, S. Ryoo, and K. Yoon. Directional Texture Transfer. In Proceedings of the 8th International Symposium on Non-Photorealistic Animation and Rendering, NPAR ’10, pages 43–48, New York, NY, USA, 2010. ACM. 1
- [23] J. Long, E. Shelhamer, and T. Darrell. Fully Convolutional Networks for Semantic Segmentation. pages 3431–3440, 2015. 2
- [24] A. Mahendran and A. Vedaldi. Understanding Deep Image Representations by Inverting Them. arXiv:1412.0035 [cs], Nov. 2014. arXiv: 1412.0035. 3, 6
- [25] J. Portilla and E. P. Simoncelli. A Parametric Texture Model Based on Joint Statistics of Complex Wavelet Coefficients. International Journal of Computer Vision, 40(1):49–70, Oct. 2000. 3, 4
- [26] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei. ImageNet Large Scale Visual Recognition Challenge. arXiv:1409.0575 [cs], Sept. 2014. arXiv: 1409.0575. 3
- [27] K. Simonyan, A. Vedaldi, and A. Zisserman. Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps. arXiv:1312.6034 [cs], Dec. 2013. 3
- [28] K. Simonyan and A. Zisserman. Very Deep Convolutional Networks for Large-Scale Image Recognition. arXiv:1409.1556 [cs], Sept. 2014. arXiv: 1409.1556. 3
- [29] J. B. Tenenbaum and W. T. Freeman. Separating style and content with bilinear models. Neural computation, 12(6):1247–1283, 2000. 2
- [30] L. Wei and M. Levoy. Fast texture synthesis using tree-structured vector quantization. In Proceedings of the 27th annual conference on Computer graphics and interactive techniques, pages 479–488. ACM Press/Addison-Wesley Publishing Co., 2000. 1
- [31] D. L. K. Yamins, H. Hong, C. F. Cadieu, E. A. Solomon, D. Seibert, and J. J. DiCarlo. Performance-optimized hierarchical models predict neural responses in higher visual cortex. Proceedings of the National Academy of Sciences, page 201403112, May 2014. 8
- 2422
- [32] C. Zhu, R. H. Byrd, P. Lu, and J. Nocedal. Algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound-constrained optimization. ACM Transactions on Mathematical Software (TOMS), 23(4):550–560, 1997. 6
- 2423

these are my assignment instructions

A3: Style Transfer

Due: Sun Apr 26, 2026 11:59pm Due: Sun Apr 26, 2026 11:59pm

ungraded

Attempt

Attempt 1

In Progress

NEXT UP: Submit Assignment

Unlimited Attempts Allowed

Tasks

[40%] Part 1: follow Style_Transfer_Exercise.ipynb from the Jupyter Notebooks section within Course Resources, complete all the TODOs in there (you may need to read the original paper, link is in both the lecture slides and the notebook)

[20%] Part 2: play with as many hyper parameter settings as you can to experiment and compare the outcome

[40%] Write an report summarizing your implementation and findings. Include any visualizations you think helpful for the analysis of your implementation and findings in the report. Submit it as a separate PDF file (Put important code snippet in the report)

Submission Requirements

[If not done yet] Create a Github repository that includes the following:

[If not done yet] Name of your repository:

CSCI611_Spring26_<Firstname>_<Lastname>

Within your repository, create project or folder: Assignment_3

Within this project or folder, include the following:

A readme file describes how to use your code

Source code in Jupyter Notebook.

Execution trace of the code in Jupyter Notebook with its generated outputs, graphs and tables etc.

A PDF file of the report

Any additional data, tools, support material that are used in the project

Submission (both items):

In Canvas Assignments, submit the PDF report as File Upload

In Canvas Assignments, submit the link to your Github repo/projects as Comments.

Additional Notes:

For submission of Github links, you need to either make your Github repo public, or add me as a collaborator. Here is an instruction on adding people as collaborators.

<https://docs.github.com/en/account-and-profile/setting-up-and-managing-your-personal-account-on-github/managing-access-to-your-personal-repositories/inviting-collaborators-to-a-personal-repository>Links to an external site.

You can find me on Github using my email: bshen2@csuchico.edu or my Github account name: boshen-csuchico.

Choose a submission type

attached is my jupyter lab notebook

can you please walk me through step by step how to complete this assignment so i can learn as a i go and first we need to complete the TODO portion then modify as many hyper parameters as possible to complete the report

I'm going to inspect your notebook, map each TODO to the paper, and then turn that into a step-by-step completion guide you can follow in order.

I found the notebook TODOs. There are really only three coding pieces to finish: image paths, the VGG layer mapping, the Gram matrix, and the training-loop losses. After that, the rest of the assignment is mostly structured experimentation and writing up what happened.

Thought for 26s >

Yes — let's do this in the same order your assignment expects:

1. **finish the notebook TODOs**
2. **run controlled hyperparameter experiments**
3. **turn the results into a report + repo submission**

I read your notebook, and the missing parts are very standard for Gatys style transfer. I'll walk you through exactly what to put in, why it works, and how to organize the experiments so your report is easy to write.

1) What the paper is doing in plain English

The core idea of the Gatys paper is:

- A pretrained **VGG19** network already contains useful visual representations.
- **Content** is represented by activations from a deeper layer, usually `conv4_2`.
- **Style** is represented by **Gram matrices** of activations from several layers like:
 - `conv1_1`
 - `conv2_1`
 - `conv3_1`
 - `conv4_1`
 - `conv5_1`

Why Gram matrices?

Because they measure **feature correlations**, which capture texture/pattern/style without preserving exact object layout.

Then the algorithm creates a target image and optimizes it so that:

- its deep features match the **content image**
- its Gram matrices match the **style image**

So the total loss is:

$$L_{\text{total}} = \alpha L_{\text{content}} + \beta L_{\text{style}}$$

This is exactly what your notebook is asking you to implement.

2) Complete the TODOs in the notebook

TODO A: choose your images

Your notebook currently has:

```
Python
content = load_image('my.content.image.jpg').to(device)
style = load_image('m.style.image.jpg', shape=content.shape[-2:]).to(device)
```

Replace those file names with your real images.

Example:

Python



Run

```
content = load_image('images/tubingen.jpg').to(device)
style = load_image('images/starry_night.jpg', shape=content.shape[-2:]).to(device)
```

What to do

- Put your images in the same folder as the notebook, or in a subfolder like `images/`
- Use one **photo** as content
- Use one **painting/art image** as style

Good image choices

- Content: city, building, portrait, landscape
- Style: Van Gogh, Picasso, Munch, Kandinsky, Monet, etc.

Tip

Start with smaller images while debugging. The notebook already uses `max_size=400`, which is good.

TODO B: fill in the VGG layer mapping

Your notebook has:

Python



Run

```
def get_features(image, model, layers=None):
    """ Run an image forward through a model and get the features for
        a set of layers. Default layers are for VGGNet matching Gatys et al (2016)
    """

    ## TODO: Complete mapping layer names of PyTorch's VGGNet to names from the paper
    ## Need the layers for the content and style representations of an image
    if layers is None:
        layers = {'0': 'conv1_1'}
```

Replace that with:

Python



Run

```
def get_features(image, model, layers=None):
    """ Run an image forward through a model and get the features for
        a set of layers. Default layers are for VGGNet matching Gatys et al (2016)
    """

    if layers is None:
        layers = {
            '0': 'conv1_1',
            '5': 'conv2_1',
            '10': 'conv3_1',
            '19': 'conv4_1',
            '21': 'conv4_2', # content representation
            '28': 'conv5_1'
        }

    features = {}
    x = image
    for name, layer in model._modules.items():
        x = layer(x)
        if name in layers:
            features[layers[name]] = x

    return features
```

Why these indices?

In PyTorch VGG19 features, the conv layers appear in sequence. The standard mapping used in many style transfer notebooks is:

- 0 → conv1_1
- 5 → conv2_1
- 10 → conv3_1
- 19 → conv4_1
- 21 → conv4_2
- 28 → conv5_1

This matches the paper's content/style choices.

TODO C: implement the Gram matrix

Your notebook has:

```
<> Python Run
def gram_matrix(tensor):
    """ Calculate the Gram Matrix of a given tensor
        Gram Matrix: https://en.wikipedia.org/wiki/Gramian_matrix
    """

    ## get the batch_size, depth, height, and width of the Tensor
    ## reshape it, so we're multiplying the features for each channel
    ## calculate the gram matrix
    gram = None

    return gram
```

Replace it with:

```
<> Python Run
def gram_matrix(tensor):
    """ Calculate the Gram Matrix of a given tensor """

    batch_size, d, h, w = tensor.size()
    tensor = tensor.view(d, h * w)
    gram = torch.mm(tensor, tensor.t())

    return gram
```

Why this works

Suppose a feature map has shape:

- depth = number of channels/features
- height × width = spatial positions

Reshaping to (d, h*w) turns each channel into a long vector.

Then:

```
<> Python Run
torch.mm(tensor, tensor.t())
```

computes the inner products between channels. That gives the feature correlation matrix, which is the style representation.

TODO D: implement target features, content loss, style loss, total loss

Your notebook has this unfinished training loop:

```

Python
for ii in range(1, steps+1):

    ## TODO: get the features from your target image
    ## Then calculate the content loss
    target_features = None
    content_loss = None

    # the style loss
    style_loss = 0
    for layer in style_weights:
        target_feature = target_features[layer]
        _, d, h, w = target_feature.shape

        ## TODO: Calculate the target gram matrix
        target_gram = None

        ## TODO: get the "style" style representation
        style_gram = None
        ## TODO: Calculate the style loss for one layer, weighted appropriately
        layer_style_loss = None

        style_loss += layer_style_loss / (d * h * w)

    ## TODO: calculate the *total* loss
    total_loss = None

```

Replace it with:

```

Python
for ii in range(1, steps+1):

    # get the features from the target image
    target_features = get_features(target, vgg)

    # content loss
    content_loss = torch.mean((target_features['conv4_2'] - content_features['conv4_2'])*

    # style loss
    style_loss = 0
    for layer in style_weights:
        target_feature = target_features[layer]
        _, d, h, w = target_feature.shape

        # target gram matrix
        target_gram = gram_matrix(target_feature)

        # style gram matrix
        style_gram = style_grams[layer]

        # layer style loss
        layer_style_loss = style_weights[layer] * torch.mean((target_gram - style_gram)**

        # normalized accumulation
        style_loss += layer_style_loss / (d * h * w)

    # total loss
    total_loss = content_weight * content_loss + style_weight * style_loss

    optimizer.zero_grad()
    total_loss.backward()
    optimizer.step()

    if ii % show_every == 0:
        print('Total loss: ', total_loss.item())

```

```
plt.imshow(im_convert(target))  
plt.show()
```

3) Why each loss term looks like that

Content loss

Python



Run

```
content_loss = torch.mean((target_features['conv4_2'] - content_features['conv4_2'])**2)
```

This measures whether the target image preserves the deeper structure of the content image.

- small value → target keeps the same high-level scene/object layout
- large value → target drifts away from the content

Style loss

For each style layer:

Python



Run

```
target_gram = gram_matrix(target_feature)  
style_gram = style_grams[layer]  
layer_style_loss = style_weights[layer] * torch.mean((target_gram - style_gram)**2)
```

This says:

- compute feature correlations for the current target
- compare them to those of the style image
- weight the importance of that layer

Then:

Python



Run

```
style_loss += layer_style_loss / (d * h * w)
```

normalizes by layer size so large layers do not dominate too aggressively.

Total loss

Python



Run

```
total_loss = content_weight * content_loss + style_weight * style_loss
```

This is the main tradeoff:

- bigger `style_weight` → more stylized image
- bigger `content_weight` → more faithful to original content

4) First run: get one correct result before experimenting

Do this first.

Recommended first settings

Use the notebook defaults:

```
Python

style_weights = {'conv1_1': 1.0,
                  'conv2_1': 0.8,
                  'conv3_1': 0.5,
                  'conv4_1': 0.3,
                  'conv5_1': 0.1}

content_weight = 1
style_weight = 1e6
optimizer = optim.Adam([target], lr=0.003)
steps = 2000
```

What you should expect

- At early iterations: image looks noisy
- Midway: style texture starts appearing
- Later: clearer stylization

If it fails badly:

- reduce image size
- reduce steps for testing
- verify file paths
- verify the layer mapping is exactly correct

5) Then do Part 2: systematic hyperparameter experiments

Your instructor wants you to “play with as many hyperparameter settings as you can.”

Do not do random guessing only. Make it look organized.

Use a small experiment table like this.

Experiment group A: content/style ratio

Keep everything else fixed.

Try:

- style_weight = 1e5
- style_weight = 5e5
- style_weight = 1e6
- style_weight = 5e6
- style_weight = 1e7

with:

```
Python

content_weight = 1
```

What to observe

- Low style weight: weak stylization, content preserved strongly
- High style weight: stronger texture/colors, but can distort objects

Save outputs with labels

Example filenames:

- exp_ratio_1e5.png
- exp_ratio_1e6.png

- `exp_ratio_1e7.png`

Experiment group B: number of steps

Keep weights fixed and vary iterations.

Try:

- `steps = 300`
- `steps = 800`
- `steps = 1500`
- `steps = 3000`

What to observe

- Fewer steps: incomplete stylization
- More steps: more refined textures, but more compute time
- Too many steps can sometimes overcook the image depending on settings

Experiment group C: learning rate

Keep `steps` and weights fixed.

Try:

- `lr = 0.001`
- `lr = 0.003`
- `lr = 0.01`

What to observe

- Too small: slow change
- Too large: unstable or ugly artifacts
- Moderate: smoother convergence

Change this line:

Python

```
optimizer = optim.Adam([target], lr=0.003)
```



Run

Experiment group D: style layer weights

This is a strong report section because it connects directly to the paper.

Current style weights:

Python

```
style_weights = {'conv1_1': 1.0,  
                 'conv2_1': 0.8,  
                 'conv3_1': 0.5,  
                 'conv4_1': 0.3,  
                 'conv5_1': 0.1}
```



Run

Try 3 presets

Preset 1: early-layer heavy

Python



Run

```
style_weights = {'conv1_1': 1.0,
                  'conv2_1': 1.0,
                  'conv3_1': 0.3,
                  'conv4_1': 0.1,
                  'conv5_1': 0.0}
```

Expected:

- stronger coarse texture
- bolder brush-like patterns

Preset 2: balanced

Python



Run

```
style_weights = {'conv1_1': 1.0,
                  'conv2_1': 0.8,
                  'conv3_1': 0.5,
                  'conv4_1': 0.3,
                  'conv5_1': 0.1}
```

Expected:

- most visually balanced

Preset 3: deeper-layer heavy

Python



Run

```
style_weights = {'conv1_1': 0.1,
                  'conv2_1': 0.2,
                  'conv3_1': 0.5,
                  'conv4_1': 0.8,
                  'conv5_1': 1.0}
```

Expected:

- less obvious local brush texture
- more abstract large-scale style structure

Experiment group E: content layer choice

This is another excellent paper-based experiment.

In the paper, deeper content layers preserve structure but allow more stylization.

Your notebook uses `conv4_2`.

Try changing content loss to compare:

Version 1:

Python



Run

```
content_loss = torch.mean((target_features['conv4_2'] - content_features['conv4_2'])*2)
```

Version 2:

Use a shallower content layer such as `conv2_1` or `conv3_1`.

To do that, first add it to your `layers` dictionary if needed. Since `conv2_1` already exists, you can use it directly.

Try:

</> Python

Run

```
content_loss = torch.mean((target_features['conv2_1'] - content_features['conv2_1'])*2)
```

Expected:

- shallower content layer → preserves more local details, can look like style overlaid on photo
- deeper content layer → better “merged” style transfer

This connects very directly to the paper’s discussion around lower vs higher content layers.

Experiment group F: target initialization

This is a nice optional experiment.

Your notebook initializes with:

</> Python

Run

```
target = content.clone().requires_grad_(True).to(device)
```

Try:

Content initialization

</> Python

Run

```
target = content.clone().requires_grad_(True).to(device)
```

Random initialization

</> Python

Run

```
target = torch.randn_like(content, device=device).requires_grad_(True)
```

Expected:

- content initialization tends to converge faster and preserve scene better
- noise initialization may be more diverse but noisier/slower

The paper discusses initialization too, so this is a strong choice.

6) How to record your experiments properly

Create a small log table while you run.

Example:

Exp ID	Content Img	Style Img	Content Layer	Style Weight	LR	Steps	Style Sun
E1	city	starry	conv4_2	1e6	0.003	2000	balae
E2	city	starry	conv4_2	1e7	0.003	2000	balae
E3	city	starry	conv2_1	1e6	0.003	2000	balae
E4	city	starry	conv4_2	1e6	0.01	2000	balae

Exp ID	Content Img	Style Img	Content Layer	Style Weight	LR	Steps	Styl Sun
E5	city	starry	conv4_2	1e6	0.003	800	bal

This table makes your report much easier.

7) Save your final images for the report

Add this helper code after displaying the final target image:

</> Python

```
plt.imshow("final_stylized.png", im_convert(target))
```

Run

For multiple experiments, save each image with a unique name:

</> Python

```
plt.imshow("outputs/exp_ratio_1e6.png", im_convert(target))
```

Run

You can create an `outputs/` folder first.

8) What to put in the report

Your report should not just say "I changed parameters."

It should explain:

- what you implemented
- why it works
- what you changed
- what happened
- what you learned

A clean report structure:

Report outline

1. Introduction

Briefly explain the assignment and paper.

Example points:

- Goal: implement neural style transfer based on Gatys et al.
- Use pretrained VGG19 features
- Preserve content from one image and style from another
- Optimize a target image using content and style losses

2. Method / Implementation

Explain the algorithm in your own words.

Include:

- VGG19 feature extractor
- chosen content layer (`conv4_2`)

- chosen style layers (conv1_1 to conv5_1)
- Gram matrix definition
- loss functions

You can include formulas:

$$L_{\text{content}} = \frac{1}{2} \sum_{ij} (F_{ij}^l - P_{ij}^l)^2$$

$$G_{ij}^l = \sum_k F_{ik}^l F_{jk}^l$$

$$L_{\text{style}} = \sum_l w_l E_l$$

$$L_{\text{total}} = \alpha L_{\text{content}} + \beta L_{\text{style}}$$

Then include a few important code snippets.

Good code snippets to include

1. `get_features()`
 2. `gram_matrix()`
 3. the main loss computation inside the loop
-

3. Experimental Setup

State:

- content image(s)
 - style image(s)
 - image size
 - optimizer
 - learning rate
 - number of steps
 - device used (CPU/GPU)
-

4. Results and Analysis

This should be the largest section.

Break it into subsections:

- effect of `style_weight`
- effect of `steps`
- effect of learning rate
- effect of style-layer weighting
- effect of content layer
- optional: effect of initialization

For each subsection:

- show 2–4 images
- explain what changed visually
- connect the result to the paper's intuition

Example sentence:

Increasing the style weight from 10^5 to 10^7 increased texture fidelity to the style image, but also distorted the structural details of the content image.

5. Discussion

Talk honestly about what worked and what did not.

Possible points:

- Higher style weight improved artistic effect but weakened object boundaries
- Shallower content layers preserved details but looked more like texture overlay
- More steps generally improved quality but increased runtime
- Random initialization produced more variation but less stability
- Results depend strongly on image pair choice

6. Conclusion

Summarize key takeaways.

Example:

- Implemented Gatys neural style transfer successfully
- Gram matrices captured style effectively
- Best results came from balanced style weights, conv4_2 content layer, and moderate learning rate
- Hyperparameters strongly affect quality and runtime

9) A simple report template you can adapt

You can use this skeleton directly.

Title

Neural Style Transfer using Convolutional Neural Networks

Abstract

This project implements the neural style transfer method proposed by Gatys et al. using a pretrained VGG19 network in PyTorch. The method separates image content and style by representing content with deep convolutional features and style with Gram matrices of feature activations across multiple layers. I completed the provided notebook, generated stylized images, and explored the effects of several hyperparameters including style/content weight ratio, learning rate, number of optimization steps, style layer weights, and content layer choice. The experiments show that the balance between content preservation and stylization is highly sensitive to these settings.

1. Introduction

Neural style transfer synthesizes an image that preserves the semantic structure of one image while adopting the visual style of another. Gatys et al. showed that pretrained CNNs learn representations where content and style can be approximately separated. This project reproduces that method and analyzes how different hyperparameters affect the final output.

2. Method

- VGG19 feature extractor
- content from conv4_2
- style from conv1_1, conv2_1, conv3_1, conv4_1, conv5_1
- Gram matrix for style
- total loss formula

3. Implementation

Describe the notebook TODOs you completed:

- mapped PyTorch VGG layers to paper layer names
- implemented Gram matrix
- computed content loss, style loss, and total loss
- optimized the target image with Adam

4. Experiments

4.1 Effect of style weight

4.2 Effect of number of steps

4.3 Effect of learning rate

4.4 Effect of style-layer weighting

4.5 Effect of content layer

4.6 Effect of initialization

5. Discussion

Explain best and worst settings.

6. Conclusion

Summarize findings.

10) Github/repo structure for submission

Your assignment wants:

- repo name: CSCI611_Spring26_<Firstname>_<Lastname>
- inside it: Assignment_3/

Use this layout:

```
CSCI611_Spring26_First_Last/
├── Assignment_3/
│   ├── README.md
│   ├── Style_Transfer_Exercise.ipynb
│   ├── report.pdf
│   ├── images/
│   │   ├── content1.jpg
│   │   ├── style1.jpg
│   │   └── ...
│   ├── outputs/
│   │   ├── exp1.png
│   │   ├── exp2.png
│   │   └── ...
│   └── optional_notes.txt
```

11) What to put in the README

Keep it simple:

README contents

- project title
- short description
- requirements
- how to run the notebook
- where outputs are saved
- brief note about the report

Example:

```

<> Markdown
# Assignment 3 – Neural Style Transfer

This project implements neural style transfer based on the paper
"Image Style Transfer Using Convolutional Neural Networks" by Gatys et al.

## Requirements
- Python 3.x
- PyTorch
- torchvision
- matplotlib
- pillow
- numpy

## How to Run
1. Open `Style_Transfer_Exercise.ipynb` in Jupyter Notebook or Jupyter Lab.
2. Place your content and style images in the `images/` folder.
3. Update the image paths in the notebook.
4. Run all cells.
5. Final images and experiment outputs can be saved to the `outputs/` folder.

## Files
- `Style_Transfer_Exercise.ipynb`: source notebook
- `report.pdf`: assignment report
- `images/`: input images
- `outputs/`: generated stylized images

```

12) Best order to actually do the work

Follow this exact order.

Phase 1: finish implementation

1. Replace image paths
2. Fill in `get_features`
3. Fill in `gram_matrix`
4. Fill in the loss loop
5. Run once with default settings
6. Make sure one stylized image works

Phase 2: experiments

7. Keep the same image pair first
8. Change only one hyperparameter at a time
9. Save every result with a clear filename
10. Log results in a table

Phase 3: report

11. Insert 1–2 equations
12. Insert 3 important code snippets
13. Add experiment images with captions
14. Write short analysis under each figure/table
15. Export to PDF

Phase 4: submission

16. Clean notebook so execution trace is present
17. Put notebook, report PDF, outputs, and README in repo
18. Upload PDF to Canvas
19. Submit repo link in Canvas comments

13) Most likely mistakes to avoid

- Wrong image filenames or paths
- Forgetting `.to(device)`
- Wrong VGG layer mapping
- Using the raw feature maps instead of Gram matrices for style
- Forgetting to set `target = content.clone().requires_grad_(True).to(device)`
- Changing too many hyperparameters at once
- Not saving experiment outputs
- Writing a report with only screenshots and no explanation

14) What I recommend as your minimum strong experiment set

This is enough for a solid report.

Use one content image and one style image. Run these:

- **Baseline:** default notebook settings
- **Style weight study:** `1e5` , `1e6` , `1e7`
- **Step study:** `500` , `1500` , `3000`
- **Learning rate study:** `0.001` , `0.003` , `0.01`
- **Content layer study:** `conv2_1` vs `conv4_2`
- **Style layer weighting study:** early-heavy vs balanced vs deep-heavy
- **Initialization study:** content init vs random init

That gives you a very good report.

15) Your exact completed TODO code in one place

Here it is together so you can copy it more easily.

`get_features`

```

Python
def get_features(image, model, layers=None):
    """ Run an image forward through a model and get the features for
        a set of layers. Default layers are for VGGNet matching Gatys et al (2016)
    """

    if layers is None:
        layers = {
            '0': 'conv1_1',
            '5': 'conv2_1',
            '10': 'conv3_1',
            '19': 'conv4_1',
            '21': 'conv4_2',
            '28': 'conv5_1'
        }

    features = {}
    x = image
    for name, layer in model._modules.items():
        x = layer(x)
        if name in layers:
            features[layers[name]] = x

    return features

```

`gram_matrix`

<> Python



▶ Run

```
def gram_matrix(tensor):
    """ Calculate the Gram Matrix of a given tensor """

    batch_size, d, h, w = tensor.size()
    tensor = tensor.view(d, h * w)
    gram = torch.mm(tensor, tensor.t())

    return gram
```

training loop TODOs

<> Python



▶ Run

```
for ii in range(1, steps+1):

    target_features = get_features(target, vgg)
    content_loss = torch.mean((target_features['conv4_2'] - content_features['conv4_2'])*

    style_loss = 0
    for layer in style_weights:
        target_feature = target_features[layer]
        _, d, h, w = target_feature.shape

        target_gram = gram_matrix(target_feature)
        style_gram = style_grams[layer]
        layer_style_loss = style_weights[layer] * torch.mean((target_gram - style_gram)**

        style_loss += layer_style_loss / (d * h * w)

    total_loss = content_weight * content_loss + style_weight * style_loss

    optimizer.zero_grad()
    total_loss.backward()
    optimizer.step()

    if ii % show_every == 0:
        print('Total loss: ', total_loss.item())
        plt.imshow(im_convert(target))
        plt.show()
```

We can do this interactively: next, paste me your image filenames or the next notebook error you hit, and I'll help you finish the TODO section cell by cell.



is this an easy task, could i finish this assignment in one day? and be ready to start writing the report the same day or tomorrow

Short answer: **yes — you can absolutely finish this in one day, if you stay structured.** This assignment is more about understanding + experimentation than heavy coding.

Let me give you a realistic breakdown so you know what to expect.



Time Estimate (honest + practical)



Part 1: Completing TODOs (1.5 – 3 hours)

- Filling in the missing code: ~30–60 min
- Debugging / fixing small issues: ~30–60 min
- First successful stylized image: ~30–60 min

👉 This is the only “technical” part.

And I already gave you the exact code — so this part should be smooth.

🟡 Part 2: Experiments (2 – 4 hours, mostly passive)

This depends on your machine.

- Each run (400px image, ~1000–2000 steps):
 - GPU: ~2–10 minutes
 - CPU: ~10–30 minutes

But here's the key:

👉 You don't sit and watch it. You run it and move on.

You'll:

- run ~8–12 experiments
- save images
- jot down observations

🟢 Part 3: Report (2 – 4 hours)

Because:

- You already understand the paper (from me explaining it)
- You already have outputs

You're mostly:

- inserting images
- explaining differences
- adding a few equations + code snippets

🔥 Total realistic timeline

Task	Time
Implementation	2–3 hrs
Experiments	2–4 hrs (parallel / passive)
Report draft	2–3 hrs
Total	6–10 hours

👉 So yes — very doable in one day

👉 Or split cleanly into:

- **Today:** code + experiments
- **Tomorrow:** report (recommended)

⚠️ The only thing that can slow you down

These 3 issues:

1. Image path errors

- Wrong filenames
- Wrong folders

2. Device mismatch

- CPU vs GPU issues (`.to(device)`)

3. Wrong layer mapping

- If VGG layer indices are wrong, results look terrible

👉 But I already gave you correct code, so you're safe.

🎯 Best strategy (what I'd do if I were you)

TODAY (focused execution)

Step 1 — Finish TODOs (1–2 hrs)

- Paste the code I gave you
- Fix image paths
- Run once until you get a decent stylized output

Step 2 — Run experiments (2–4 hrs)

While they run:

- Save outputs
- Write quick notes like:
 - "style weight too high → distorted"
 - "low steps → blurry"

👉 Don't overthink yet — just collect data

TOMORROW (easy mode)

Step 3 — Write report (2–3 hrs)

Because now:

- You already have images
- You already saw the effects

So writing becomes:

"Explain what you observed"

💡 Key mindset shift

This assignment is NOT:

❌ "build a deep learning model from scratch"

It IS:

✅ "understand how style transfer works + show you can experiment with it"

👍 Final answer

- 👉 Yes — **this is a very doable 1-day assignment**
 - 👉 You can finish implementation + experiments today
 - 👉 And be ready to write the report tonight or tomorrow
-

If you want, I can:

- help you debug your first run live
- or help you pick good content/style images
- or even help you structure your report as you go

Just tell me where you're at 🍌



can you help me activate my virtual environment first

Absolutely — let's get your virtual environment running 🍌
I'll walk you through it step by step depending on your setup.

🧠 First: What are you using?

Tell me if you're unsure, but I'll cover the common cases:

- macOS / Linux (Terminal)
- Windows (Command Prompt / PowerShell)
- VS Code + Jupyter

✅ Step 1: Navigate to your project folder

In your terminal:

```
<> Bash
cd path/to/your/project
```



Example:

```
<> Bash
cd CSCI611_Spring26_YourName/Assignment_3
```



✅ Step 2: Check if a virtual environment already exists

Look for a folder like:

- venv/
- .venv/
- env/

If you see one → activate it

If not → we'll create one

If you need to CREATE a virtual environment

Run:

```
<> Bash
python3 -m venv venv
```



This creates a folder called `venv`

Step 3: Activate the environment

On macOS / Linux:

```
<> Bash  
source venv/bin/activate
```



On Windows (Command Prompt):

```
<> cmd  
venv\Scripts\activate
```



On Windows (PowerShell):

```
<> PowerShell  
venv\Scripts\Activate.ps1
```



⚠ If PowerShell blocks it, run:

```
<> PowerShell  
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```



then activate again.

Step 4: Confirm it worked

You should see something like:

```
<> Bash  
(venv) your-computer-name %
```



That `(venv)` means you're inside the environment 

Step 5: Install required packages

Run:

```
<> Bash  
pip install torch torchvision matplotlib pillow numpy
```



If using Jupyter:

```
<> Bash  
pip install notebook ipykernel
```



Step 6: Connect it to Jupyter (important)

Run:

```
</> Bash
python -m ipykernel install --user --name=venv --display-name "Python (venv)"
```

Now when you open your notebook:

- Select kernel → **Python (venv)**

Step 7: Launch Jupyter

```
</> Bash
jupyter notebook
```

or

```
</> Bash
jupyter lab
```

Common issues

"command not found: python"

Try:

```
</> Bash
python3
```

torch install fails

Let me know your system — I'll give you the exact install command.

Kernel not showing

Restart Jupyter after running `ipykernel install`

Quick sanity check

Once activated, run:

```
</> Bash
python -c "import torch; print(torch.__version__)"
```

If it prints a version → you're good 

If you want faster help

Tell me:

- your OS (Mac / Windows)
- where you're running (VS Code / terminal / Jupyter)

and I'll tailor it exactly to your setup so you don't get stuck.



I completed

Next, I'm loading in images by file name and forcing the style image to be the same size as the content image.

TODO: use your content vs style images.

```
# load in content and style image
content = load_image('houseContent.jpg').to(device)
# Resize style to match content, makes code easier
style = load_image('vanGoghStyle.png',
shape=content.shape[-2:]).to(device)

# helper function for un-normalizing an image
# and converting it from a Tensor image to a NumPy image for display
def im_convert(tensor):
    """ Display a tensor as an image. """

    image = tensor.to("cpu").clone().detach()
    image = image.numpy().squeeze()
    image = image.transpose(1,2,0)
    image = image * np.array((0.229, 0.224, 0.225)) + np.array((0.485,
0.456, 0.406))
    image = image.clip(0, 1)

    return image

# display the images
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 10))
# content and style ims side-by-side
ax1.imshow(im_convert(content))
ax2.imshow(im_convert(style))
```

now im here and i think ive reached my first todo

VGG19 Layers

To get the content and style representations of an image, we have to pass an image forward through the VGG19 network until we get to the desired layer(s) and then get the output from that layer.

```
# print out VGG19 structure so you can see the names of various
layers
# print(vgg)
```

then theres another todo here

Content and Style Features

TODO: complete the mapping of layer names to the names found in the paper for the content representation and the style representation. The first layer (0) to conv1_1 has been done for you, below.

```
def get_features(image, model, layers=None):
    """ Run an image forward through a model and get the features for
        a set of layers. Default layers are for VGGNet matching Gatys et
        al (2016)
    """

    ## TODO: Complete mapping layer names of PyTorch's VGGNet to
    names from the paper
    ## Need the layers for the content and style representations of an
    image
    if layers is None:
        layers = {'0': 'conv1_1'}

    ## -- do not need to change the code below this line -- ##
    features = {}
    x = image
    # model._modules is a dictionary holding each module in the model
    for name, layer in model._modules.items():
        x = layer(x)
        if name in layers:
            features[layers[name]] = x

    return features
```

what do i do step by step here ?? what ideas do you have and have i reached my next todo? yea right?

Thought for a couple of seconds

Yes — you've reached the next TODO exactly.

You already finished the **image-loading TODO**, and now you are at the **VGG layer mapping TODO**.

This next step is important because the paper refers to layers by names like:

- conv1_1
- conv2_1
- conv3_1
- conv4_1
- conv4_2
- conv5_1

But PyTorch's VGG model uses numeric indices like '0', '5', '10', etc.

So your job is to create a dictionary that says:

"When I see PyTorch layer '0', treat it as paper layer conv1_1."

What you should do right now

Replace this:

```
<> Python
if layers is None:
    layers = {'0': 'conv1_1'}
```



Run

with this:

Python



Run

```
if layers is None:
    layers = {
        '0': 'conv1_1',
        '5': 'conv2_1',
        '10': 'conv3_1',
        '19': 'conv4_1',
        '21': 'conv4_2',
        '28': 'conv5_1'
    }
```

So your full function becomes:

Python



Run

```
def get_features(image, model, layers=None):
    """ Run an image forward through a model and get the features for
        a set of layers. Default layers are for VGGNet matching Gatys et al (2016)
    """

    if layers is None:
        layers = {
            '0': 'conv1_1',
            '5': 'conv2_1',
            '10': 'conv3_1',
            '19': 'conv4_1',
            '21': 'conv4_2',
            '28': 'conv5_1'
        }

    ## -- do not need to change the code below this line -- ##
    features = {}
    x = image
    for name, layer in model._modules.items():
        x = layer(x)
        if name in layers:
            features[layers[name]] = x

    return features
```

Why these exact layers?

The paper uses:

- **content** from conv4_2
- **style** from:
 - conv1_1
 - conv2_1
 - conv3_1
 - conv4_1
 - conv5_1

So this dictionary gives you access to all of them.

Step by step: what this function is doing

Step 1

You send the image through VGG19 layer by layer:

Python



Run

```
x = layer(x)
```

Step 2

Whenever the current layer index is one you care about, save it:

```
<> Python
```



```
▶ Run
```

```
if name in layers:
    features[layers[name]] = x
```

That means:

- if name == '0' , save as features['conv1_1']
- if name == '21' , save as features['conv4_2']

Step 3

Return all those saved feature maps in a dictionary.

So later you'll be able to write things like:

```
<> Python
```



```
▶ Run
```

```
content_features['conv4_2']
style_features['conv1_1']
```

What idea should you remember here?

This is the main concept:

- **The CNN is already pretrained**
- You are **not training VGG**
- You are using VGG as a **feature extractor**

And different layers capture different kinds of information:

- **lower layers** → edges, colors, textures
- **higher layers** → objects, structure, layout

That is why:

- style uses multiple layers
- content usually uses a deeper layer like conv4_2

Yes, you are at the next TODO

Your sequence so far is:

1. load images ✓
2. display images ✓
3. reach get_features() TODO ✓
4. fill layer mapping 🔄 **this is what you do now**

After you fill this in, what should you do next?

Run the cell.

Then the notebook will probably move to something like:

- extracting content and style features
- computing Gram matrices
- another TODO for gram_matrix()

That will be your next coding step after this one.

What you can do right after this cell

After running it, test it with:

<> Python

Copy

Run

```
content_features = get_features(content, vgg)
style_features = get_features(style, vgg)

print(content_features.keys())
print(style_features.keys())
```

You should see something like:

<> Python

Copy

Run

```
dict_keys(['conv1_1', 'conv2_1', 'conv3_1', 'conv4_1', 'conv4_2', 'conv5_1'])
```

If you see that, this TODO is done correctly.

What to write in your own notes for learning

You can write:

I mapped PyTorch VGG19 layer indices to the layer names used in the Gatys paper so I could extract the correct content and style representations. The content representation uses conv4_2 , while the style representation uses Gram matrices from multiple layers: conv1_1 through conv5_1 .

That will help later in your report.

Paste the next TODO after this one and I'll walk you through it the same way.

Copy

Like

Dislike

Share

Refresh

More

great, now im on this step

Gram Matrix
The output of every convolutional layer is a Tensor with dimensions associated with the batch_size, a depth, d and some height and width (h, w). The Gram matrix of a convolutional layer can be calculated as follows:

Get the depth, height, and width of a tensor using batch_size, d, h, w = tensor.size()
Reshape that tensor so that the spatial dimensions are flattened
Calculate the gram matrix by multiplying the reshaped tensor by it's transpose
Note: You can multiply two matrices using torch.mm(matrix1, matrix2).

TODO: Complete the gram_matrix function.

```
def gram_matrix(tensor):
    """ Calculate the Gram Matrix of a given tensor
```

Gram Matrix: https://en.wikipedia.org/wiki/Gramian_matrix

```
"""
```

```
## get the batch_size, depth, height, and width of the Tensor
## reshape it, so we're multiplying the features for each channel
## calculate the gram matrix
gram = None
```

```
return gram
```

according to the notes, a gram matrix is when we

1. flatten each layer
2. put flattened layers in a matrix
3. multiply original matrix by the transpose of that matrix (rows become columns? something like that double check)

then we get the eventual gram matrix to keep track of each layer

can you please help me step by step what to I change here?

Thought for a couple of seconds

Yes — your understanding is basically right.

A tiny correction:

- you are **not flattening each layer of the whole network**
- you are flattening the **feature maps inside one convolutional layer output**

So for one layer output tensor shaped like:

$$(batch_size, d, h, w)$$

you turn it into something like:

$$(d, h \times w)$$

Then you multiply that matrix by its transpose:

$$G = FF^T$$

That gives the **Gram matrix**, which measures how similar each feature map is to every other feature map.

So yes:

- rows become columns in the transpose
- the result is a matrix of **feature correlations**

What to change

Replace this:

Python



Run

```
def gram_matrix(tensor):
    """ Calculate the Gram Matrix of a given tensor
    Gram Matrix: https://en.wikipedia.org/wiki/Gramian_matrix
    """

    ## get the batch_size, depth, height, and width of the Tensor
    ## reshape it, so we're multiplying the features for each channel
```

```
## calculate the gram matrix
gram = None

return gram
```

with this:

```
<> Python Run
def gram_matrix(tensor):
    """ Calculate the Gram Matrix of a given tensor """

    # get the batch size, depth, height, and width
    batch_size, d, h, w = tensor.size()

    # reshape so each channel is a row and each spatial location is a column
    tensor = tensor.view(d, h * w)

    # calculate the gram matrix
    gram = torch.mm(tensor, tensor.t())

    return gram
```

Step by step explanation

Step 1: get the tensor shape

```
<> Python Run
batch_size, d, h, w = tensor.size()
```

This means:

- `batch_size` : number of images in batch
- `d` : number of feature maps / channels
- `h` : feature map height
- `w` : feature map width

Usually here batch size is 1.

Step 2: reshape it

```
<> Python Run
tensor = tensor.view(d, h * w)
```

Why?

Each of the `d` channels is a 2D feature map of size `h x w`.

We flatten each feature map into one long vector of length `h*w`.

So instead of:

- `d` feature maps, each of size `h x w`

we now have:

- a matrix with shape `(d, h*w)`

That means:

- each **row** = one flattened feature map
- each **column** = one spatial position

Step 3: multiply by transpose

Python



Run

```
gram = torch.mm(tensor, tensor.t())
```

Here:

- `tensor` has shape $(d, h \times w)$
- `tensor.t()` has shape $(h \times w, d)$

So the result has shape:

$$(d, d)$$

That means:

- row i , column j tells you how correlated feature map i is with feature map j

That is the style representation.

Why does this capture style?

Because style is more about:

- textures
- repeated patterns
- color relationships
- brushstroke-like statistics

and less about exact object placement.

The Gram matrix ignores exact spatial arrangement and instead captures **which features tend to appear together**.

That's why it works for style transfer.

A concrete mental picture

Suppose one layer has:

- $d = 3$ feature maps
- $h = 4, w = 4$

Then after flattening, each feature map becomes length 16.

So your matrix becomes:

$$F \in \mathbb{R}^{3 \times 16}$$

Then:

$$G = FF^T$$

gives:

$$G \in \mathbb{R}^{3 \times 3}$$

Now $G[0, 1]$ tells you how similar feature map 0 is to feature map 1.

One small note

Some implementations include batch size in the reshape, like:

Python



Run

```
tensor = tensor.view(batch_size * d, h * w)
```

But in your notebook, batch size is effectively 1, so this version is correct and standard:

Python



Run

```
tensor = tensor.view(d, h * w)
```

What to do right now

1. Paste this function

Python



Run

```
def gram_matrix(tensor):
    """ Calculate the Gram Matrix of a given tensor """

    batch_size, d, h, w = tensor.size()
    tensor = tensor.view(d, h * w)
    gram = torch.mm(tensor, tensor.t())

    return gram
```

2. Run the cell

3. If the notebook has a test cell after it, run that too

You can also test it manually with something like:

Python



Run

```
features = get_features(style, vgg)
gram = gram_matrix(features['conv1_1'])
print(gram.shape)
```

You should get a square matrix, something like:

Python



Run

```
torch.Size([64, 64])
```

depending on the layer.

What to write in your notes

You can write:

For each selected style layer, I reshaped the feature maps into a matrix where each row corresponds to one channel and each column corresponds to one spatial location. I then multiplied this matrix by its transpose to compute the Gram matrix, which captures feature correlations and serves as the style representation.

Yes — this TODO is the correct next one, and once you finish it, the next step will probably be computing:

- style_grams
- content loss
- style loss

- total loss

Paste the next section when you reach it and I'll walk you through that one too.



great, now I'm on this section

Updating the Target & Calculating Losses

You'll decide on a number of steps for which to update your image, this is similar to the training loop that you've seen before, only we are changing our target image and nothing else about VGG19 or any other image. Therefore, the number of steps is really up to you to set! I recommend using at least 2000 steps for good results. But, you may want to start out with fewer steps if you are just testing out different weight values or experimenting with different images.

Inside the iteration loop, you'll calculate the content and style losses and update your target image, accordingly.

Content Loss

The content loss will be the mean squared difference between the target and content features at layer conv4_2. This can be calculated as follows:

```
content_loss = torch.mean((target_features['conv4_2'] -
content_features['conv4_2'])**2)
```

Style Loss

The style loss is calculated in a similar way, only you have to iterate through a number of layers, specified by name in our dictionary style_weights.

You'll calculate the gram matrix for the target image, target_gram and style image style_gram at each of these layers and compare those gram matrices, calculating the layer_style_loss. Later, you'll see that this value is normalized by the size of the layer.

Total Loss

Finally, you'll create the total loss by adding up the style and content losses and weighting them with your specified alpha and beta!

Intermittently, we'll print out this loss; don't be alarmed if the loss is very large. It takes some time for an image's style to change and you should focus on the appearance of your target image rather than any loss value. Still, you should see that this loss decreases over some number of iterations.

TODO: Define content, style, and total losses.

```
# for displaying the target image, intermittently
show_every = 400
```

```
# iteration hyperparameters
optimizer = optim.Adam([target], lr=0.003)
steps = 2000 # decide how many iterations to update your image
(5000)
```

```
for ii in range(1, steps+1):
```

```

## TODO: get the features from your target image
## Then calculate the content loss
target_features = None
content_loss = None

# the style loss
# initialize the style loss to 0
style_loss = 0
# iterate through each style layer and add to the style loss
for layer in style_weights:
    # get the "target" style representation for the layer
    target_feature = target_features[layer]
    _, d, h, w = target_feature.shape

    ## TODO: Calculate the target gram matrix
    target_gram = None

    ## TODO: get the "style" style representation
    style_gram = None
    ## TODO: Calculate the style loss for one layer, weighted
    appropriately
    layer_style_loss = None

    # add to the style loss
    style_loss += layer_style_loss / (d * h * w)

## TODO: calculate the *total* loss
total_loss = None

## -- do not need to change code, below -- ##
# update your target image
optimizer.zero_grad()
total_loss.backward()
optimizer.step()

# display intermediate images and print the loss
if ii % show_every == 0:
    print('Total loss: ', total_loss.item())
    plt.imshow(im_convert(target))
    plt.show()

```

can you please help me step by step, what do i put in this section??

You're at the last major coding TODO. This one just wires together the pieces you already built: `get_features` , `gram_matrix` , the saved `content_features` , and the saved `style_grams` .

Yes — this is the last main coding TODO, and it's the part where everything comes together.

You already have:

- image loading
- feature extraction
- Gram matrix

Now you use those to compute:

- **content loss**
- **style loss**
- **total loss**

- then update the target image

What to put in this section

Replace the `None` values with this:

```
# for displaying the target image, intermittently
show_every = 400

# iteration hyperparameters
optimizer = optim.Adam([target], lr=0.003)
steps = 2000 # decide how many iterations to update your image (5000)

for ii in range(1, steps+1):

    # get the features from your target image
    target_features = get_features(target, vgg)

    # calculate the content loss
    content_loss = torch.mean((target_features['conv4_2'] - content_features['conv4_2'])*

    # the style loss
    style_loss = 0

    # iterate through each style layer and add to the style loss
    for layer in style_weights:
        # get the "target" style representation for the layer
        target_feature = target_features[layer]
        _, d, h, w = target_feature.shape

        # calculate the target gram matrix
        target_gram = gram_matrix(target_feature)

        # get the "style" style representation
        style_gram = style_grams[layer]

        # calculate the style loss for one layer, weighted appropriately
        layer_style_loss = style_weights[layer] * torch.mean((target_gram - style_gram)**

        # add to the style loss
        style_loss += layer_style_loss / (d * h * w)

    # calculate the total loss
    total_loss = content_weight * content_loss + style_weight * style_loss

    ## -- do not need to change code, below -- ##
    # update your target image
    optimizer.zero_grad()
    total_loss.backward()
    optimizer.step()

    # display intermediate images and print the loss
    if ii % show_every == 0:
        print('Total loss: ', total_loss.item())
        plt.imshow(im_convert(target))
        plt.show()
```

Step by step: what each line means

1) Get target features

```
target_features = get_features(target, vgg)
```

This sends your current target image through VGG.

At the start, the target is probably just a copy of the content image or random noise.

As optimization runs, the target image changes.

You need to recalculate its features every iteration because the image is being updated.

2) Content loss

```
content_loss = torch.mean((target_features['conv4_2'] - content_features['conv4_2'])**2)
```

This compares:

- the target image's content features
- the original content image's content features

Both at layer conv4_2 .

Why conv4_2 ?

Because the paper uses a deeper layer for content so the algorithm preserves high-level structure rather than exact pixels.

What this means

If this loss is small, the target image still "looks like" the content image in terms of scene/object layout.

3) Initialize style loss

```
style_loss = 0
```

You'll add style loss from multiple style layers.

4) Loop over each style layer

```
for layer in style_weights:
```

This goes through something like:

```
{
  'conv1_1': 1.0,
  'conv2_1': 0.8,
  'conv3_1': 0.5,
  'conv4_1': 0.3,
  'conv5_1': 0.1
}
```

Each layer contributes some amount to the style loss.

5) Get current target features for that layer

```
target_feature = target_features[layer]
_, d, h, w = target_feature.shape
```

This gives you the target's activations at that specific style layer.

You also extract:

- `d` = number of channels
- `h, w` = spatial size

These are used for normalization later.

6) Compute target Gram matrix

```
target_gram = gram_matrix(target_feature)
```

This converts the target's feature map into a style representation.

7) Get precomputed style Gram matrix

```
style_gram = style_grams[layer]
```

Earlier in the notebook, you should already have something like:

```
style_features = get_features(style, vgg)
style_grams = {layer: gram_matrix(style_features[layer]) for layer in style_features}
```

So here you are comparing:

- current target style
- desired style image style

8) Compute style loss for one layer

```
layer_style_loss = style_weights[layer] * torch.mean((target_gram - style_gram)**2)
```

This measures how different the target's style is from the style image at one layer.

Then it multiplies that by the weight for that layer.

Why layer weights?

Because not all layers contribute equally to style.

Lower layers capture smaller textures; higher layers capture larger patterns.

9) Normalize and accumulate

```
style_loss += layer_style_loss / (d * h * w)
```

This keeps larger layers from dominating too much.

Without normalization, layers with more values could contribute way more just because they are bigger.

10) Total loss

```
total_loss = content_weight * content_loss + style_weight * style_loss
```

This is the key tradeoff.

- `content_weight` controls how much structure you preserve
- `style_weight` controls how much stylization you get

A common default is:

```
content_weight = 1
style_weight = 1e6
```

What this loop is doing overall

Every iteration:

1. run target image through VGG
2. compare target content to content image
3. compare target style to style image
4. combine both losses
5. backpropagate
6. adjust pixels of the target image

So you're not training the network.

You are training the **image pixels**.

That's the key idea.

What you should check before running this

Make sure these were already defined earlier:

content features

```
content_features = get_features(content, vgg)
```

style features

```
style_features = get_features(style, vgg)
```

style grams

```
style_grams = {layer: gram_matrix(style_features[layer]) for layer in style_features}
```

layer weights

```
style_weights = {'conv1_1': 1.,  
                 'conv2_1': 0.8,  
                 'conv3_1': 0.5,  
                 'conv4_1': 0.3,  
                 'conv5_1': 0.1}
```

content/style weights

```
content_weight = 1  
style_weight = 1e6
```

target image

Often:

```
target = content.clone().requires_grad_(True).to(device)
```

What result to expect

When you run this:

- at first the image may look almost unchanged or noisy
- after a few hundred iterations, style patterns should become visible
- around 2000 steps, you should get a meaningful stylized image

If it looks too weak:

- increase `style_weight`

If it looks too distorted:

- reduce `style_weight`
- or increase `content_weight`

A quick mental summary for your notes

You can write this for yourself:

In each optimization step, I extracted the current target image features using VGG19. I computed content loss as the mean squared error between the target and content image at `conv4_2`. I computed style loss by comparing Gram matrices of the target and style image across multiple layers. The total loss was a weighted sum of content and style losses, and I optimized the target image using Adam.

Yes — once this runs, your TODO section is basically done

After this:

- the core implementation is complete
- next you'll be ready to generate your first result
- then move into hyperparameter experiments for the report

What to do next

1. paste the completed code
2. run the cell
3. watch for the first image output around step 400

If you get an error, paste the exact error and I'll fix it with you.

If it runs, send me the next section and I'll help you start the experimentation phase.



finally it finished # display content and final, target image fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 10))
 ax1.imshow(im_convert(content)) ax2.imshow(im_convert(target))
 and this displays the target image correct? It looks pretty good but according to the assignment I am supposed to play with the hyper parameters or? for report purposes? A3: Style Transfer Due: Sun Apr 26, 2026 11:59pm Due: Sun Apr 26, 2026 11:59pm ungraded Attempt 1 In Progress NEXT UP: Submit Assignment Unlimited Attempts Allowed Tasks [40%] Part 1: follow Style_Transfer_Exercise.ipynb from the Jupyter Notebooks section within Course Resources, complete all the TODOs in there (you may need to read the original paper, link is in both the lecture slides and the notebook) [20%] Part 2: play with as many hyper parameter settings as you can to experiment and compare the outcome [40%] Write an report summarizing your implementation and findings. Include any visualizations you think helpful for the analysis of your implementation and findings in the report. Submit it as a separate PDF file (Put important code snippet in the report) Submission Requirements [If not done yet] Create a Github repository that includes the following: [If not done yet] Name of your repository: CSCI611_Spring26_<Firstname>_<Lastname> Within your repository, create project or folder: Assignment_3 Within this project or folder, include the following: A readme file describes how to use your code Source code in Jupyter Notebook. Execution trace of the code in Jupyter Notebook with its generated outputs, graphs and tables etc. A PDF file of the report Any additional data, tools, support material that are used in the project Submission (both items): In Canvas Assignments, submit the PDF report as File Upload In Canvas Assignments, submit the link to your Github repo/projects as Comments. Additional Notes: For submission of Github links, you need to either make your Github repo public, or add me as a collaborator. Here is an instruction on adding people as collaborators. <https://docs.github.com/en/account-and-profile/setting-up-and-managing-your-personal-account-on-github/managing-access-to-your-personal-repositories/inviting-collaborators-to-a-personal-repository> Links to an external site. You can find me on Github using my email: bshen2@csuchico.edu or my Github account name: boshen-csuchico. Choose a submission type

Thought for a couple of seconds

Yes — `ax2.imshow(im_convert(target))` is displaying your **final stylized target image**.

So at this point:

- `ax1` = original **content** image
- `ax2` = final **style transfer result**

That means **Part 1 is basically complete**: you finished the TODOs and produced a working result.

Now the assignment wants **Part 2** and **Part 3**:

- **Part 2**: experiment with hyperparameters
- **Part 3**: write the report summarizing implementation + findings

So you are in a good spot.

What you should do next

Do **not** stop at one final image.

Your instructor explicitly wants you to **play with hyperparameter settings and compare the outcome**.

That means you should now run a set of controlled experiments and save the results for your report.

What counts as “hyperparameters” here?

For this notebook, the main ones are:

1. **style weight**
2. **content weight**
3. **number of steps**
4. **learning rate**
5. **style layer weights**
6. **content layer choice**
7. **target initialization** (optional but good)

You do not necessarily need to change every possible thing in the universe, but you should change enough settings to show that you explored the method.

Best next move: do experiments in a structured way

Use one content image and one style image first, and vary **one thing at a time**.

That makes your report much stronger because you can say what caused what.

Experiment plan I recommend

Experiment 1: baseline

Keep your current successful settings as the baseline.

Example:

```
content_weight = 1
style_weight = 1e6
steps = 2000
lr = 0.003
```

Save this image.

Experiment 2–4: change style weight

This is the most important one.

Try:

```
style_weight = 1e5  
style_weight = 1e6  
style_weight = 1e7
```

Keep everything else fixed.

What you'll likely see

- 1e5 → weaker stylization, content preserved more
- 1e6 → balanced
- 1e7 → stronger style, more distortion

This gives you an easy comparison for the report.

Experiment 5–7: change steps

Try:

```
steps = 500  
steps = 1000  
steps = 2000
```

Keep the weights fixed.

What you'll likely see

- 500 → underdeveloped style
 - 1000 → decent
 - 2000 → more refined
-

Experiment 8–10: change learning rate

Try:

```
lr = 0.001  
lr = 0.003  
lr = 0.01
```

Keep other things fixed.

Likely effect

- small LR → slower but stable
 - medium LR → usually good
 - large LR → can be unstable or produce harsher artifacts
-

Experiment 11–13: style layer weights

Use 3 presets.

Balanced

```
style_weights = {'conv1_1': 1.0,  
                 'conv2_1': 0.8,  
                 'conv3_1': 0.5,  
                 'conv4_1': 0.3,  
                 'conv5_1': 0.1}
```

Early-heavy

```
style_weights = {'conv1_1': 1.0,  
                 'conv2_1': 1.0,  
                 'conv3_1': 0.3,  
                 'conv4_1': 0.1,  
                 'conv5_1': 0.0}
```

Deep-heavy

```
style_weights = {'conv1_1': 0.1,  
                 'conv2_1': 0.2,  
                 'conv3_1': 0.5,  
                 'conv4_1': 0.8,  
                 'conv5_1': 1.0}
```

This is good because it connects directly to the paper.

Experiment 14–15: content layer choice

Your current content loss uses `conv4_2`. Compare it with a shallower one.

Try:

```
content_loss = torch.mean((target_features['conv2_1'] - content_features['conv2_1'])**2)
```

versus your original:

```
content_loss = torch.mean((target_features['conv4_2'] - content_features['conv4_2'])**2)
```

Likely effect

- shallower content layer → preserves more local detail
- deeper content layer → allows more natural stylization

This is also directly supported by the paper.

Minimum strong experiment set

If you are short on time, do at least these:

- 3 style weights
- 3 step counts
- 2 content layers
- 3 style-weight presets

That is already enough for a solid report.

What to save for the report

For each experiment, save:

- the output image
- the settings used
- one or two observations

Example log:

Exp	style_weight	steps	lr	content layer	notes
B1	1e6	2000	0.003	conv4_2	balanced result
S1	1e5	2000	0.003	conv4_2	weak stylization
S2	1e7	2000	0.003	conv4_2	strong style, some distortion
T1	1e6	500	0.003	conv4_2	incomplete style
C1	1e6	2000	0.003	conv2_1	more preserved detail

This table can go directly into your report.

Save your outputs

After each experiment, save the image.

Example:

```
plt.imshow("outputs/baseline.png", im convert(target))
```

For another experiment:

```
plt.imshow("outputs/style weight 1e7.png", im convert(target))
```

Make an outputs/ folder if needed.

Suggested workflow from here

Right now

1. Save your current successful result as baseline
2. Pick 6–10 experiments
3. Run them one by one
4. Save each output
5. Write down quick notes while results are fresh

Then

6. Build the report using those results
 7. Export PDF
 8. Put notebook + outputs + report + README into repo
-

What your report should focus on

Your report should answer:

- how did you implement the paper?
- what hyperparameters did you test?
- how did the results change?
- which settings worked best?
- what did you learn?

So yes, the hyperparameter experiments are not optional fluff — they are important for both:

- **Part 2 grade**
 - **Part 3 report content**
-

A very practical way to organize the report

Use these sections:

1. Introduction

Briefly explain Gatys style transfer.

2. Implementation

Show:

- VGG layer mapping
- Gram matrix code
- loss function code

3. Experimental setup

List:

- content/style images
- optimizer
- image size
- hardware if you want
- baseline settings

4. Hyperparameter experiments

Subsections:

- style weight
- number of steps
- learning rate
- content layer
- style layer weights

5. Discussion

What settings gave best results and why.

6. Conclusion

Summarize findings.

Answer to your main question

Yes:

- the displayed target image is the final result
- but **you should now experiment with hyperparameters**
- and those experiments will become the core of your report

You are now in the right phase of the assignment.

If you want, next I can give you a **clean experiment checklist with exact cells to duplicate/change** so you can run the report experiments without getting messy.



I think i want to skip this i already have 8 additional experiments without this, 7–8. Style layer weights (optional but great)
Balanced (default)
Early-heavy

Is that okay and 8 experiments are enough?

here is what I'm testing

```
# =====
# EXPERIMENT 1: BASELINE
# style_weight = 1e6
# steps = 2000
# =====
```

```
plt.imsave("outputs/BaselineExperiment1Steps_2000StyleWeight_1e6.png", im_convert(target))
```

```
# =====
# EXPERIMENT 2: LOW STYLE
# style_weight = 1e5 (Style is Weak, content is strong)
# steps = 2000
# =====
```

```
plt.imsave("outputs/Experiment2LowStyle_Steps_2000_StyleWeight_1e5.png", im_convert(target))
```

```
# =====
# EXPERIMENT 3: HIGH STYLE
# style_weight = 1e7 (Content is Weak, style is strong)
# steps = 2000
# =====
```

```
plt.imsave("outputs/Experiment3HighStyle_Steps_2000_StyleWeight_1e7.png", im_convert(target))
```

```
# =====
# EXPERIMENT 4: LOW STEPS
# style_weight = 1e6 (baseline Style)
# steps = 300 (low steps, image looks incomplete, style is weak or partially applied, some areas look noisy or unfinished)
# =====
```

```
plt.imsave("outputs/Experiment4LowSteps_Steps_300_StyleWeight_1e6.png", im_convert(target))
```

```
# =====  
# EXPERIMENT 5: MEDIUM STEPS  
# style_weight = 1e6 (baseline Style)  
# steps = 1000 (style clearly visible, structure is still preserved, looks  
# decent, as in almost done)  
# =====
```

```
plt.imsave("outputs/Experiment5MediumSteps_Steps_1000_StyleWei  
ght_1e6.png", im_convert(target))
```

```
# =====  
# EXPERIMENT 6: SHALLOW CONTENT LAYER  
# style_weight = 1e6 (baseline Style)  
# steps = 2000 (baseline steps)  
# content layer = conv2_1 instead of default conv4_2  
# content layers  
# conv1_1 and conv_2_1 represents edges textures and fine details  
# conv3_1 and conv4_1 represents shapes and patterns  
# conv4_2 and conv5_1 represents higher features such as objects  
# and structure  
# we expect more edges to be preserved, the house sharper closer to  
# the original house content photo  
# style is weaker/less integrated  
# =====
```

```
plt.imsave("outputs/Experiment6ShallowContentLayer_conv_2_1_Ste  
ps_2000_StyleWeight_1e6.png", im_convert(target))
```

```
# =====  
# EXPERIMENT 7: MEDIUM CONTENT LAYER  
# style_weight = 1e6 (baseline Style)  
# steps = 2000 (baseline steps)  
# content layer = conv3_1 instead of default conv4_2  
# content layers  
# conv1_1 and conv_2_1 represents edges textures and fine details  
# conv3_1 and conv4_1 represents shapes and patterns  
# conv4_2 and conv5_1 represents higher features such as objects  
# and structure  
# we expect the house to be in between integrated with content and  
# style with slightly more content than style  
# style is weaker/less integrated  
# =====
```

```
plt.imsave("outputs/Experiment7MediumContentLayer_conv_3_1_Ste  
ps_2000_StyleWeight_1e6.png", im_convert(target))
```

```
# =====  
# EXPERIMENT 8: DEEP CONTENT LAYER  
# style_weight = 1e6 (baseline Style)  
# steps = 2000 (baseline steps)  
# content layer = conv5_1 instead of default conv4_2  
# content layers  
# conv1_1 and conv_2_1 represents edges textures and fine details  
# conv3_1 and conv4_1 represents shapes and patterns  
# conv4_2 and conv5_1 represents higher features such as objects  
# and structure  
# we expect more style to be preserved, the house showing more  
# depth of style  
# style is stronger/more integrated
```

=====

```
plt.imsave("outputs/Experiment8MediumContentLayer_conv_5_1_Ste
ps_2000_StyleWeight_1e6.png", im_convert(target))
```

Thought for a couple of seconds

Yes — **8 experiments is enough**, especially since they are not random and they cover multiple meaningful axes:

- baseline
- style/content tradeoff via `style_weight`
- optimization progress via `steps`
- representation choice via different **content layers**

That is already a solid experimental section for this assignment.

You do **not** need to force the optional style-layer-weight experiment if you already have enough results and you're short on time.

My honest opinion on your experiment set

Your current set is actually pretty good because it has structure:

You covered:

- **baseline** → reference point
- **low vs high style weight** → one of the most important hyperparameters
- **low vs medium steps** → optimization progress / convergence
- **shallow / medium / deep content layer** → directly tied to the paper's theory

That is strong enough to write a real analysis section.

One small thing I would improve

You said "8 additional experiments," but from what you pasted it looks like:

- Experiment 1 = baseline
- Experiments 2–8 = seven more runs

So depending on how you count:

- **8 total** experiments = totally fine
- **7 beyond baseline** = also fine

Either way, you have enough.

What matters more than adding more experiments

At this point, adding more is less important than:

- making sure each experiment actually has an image saved
- writing clear observations
- comparing them well in the report

That will help your grade more than squeezing in one more optional preset.

One thing to double-check

Make sure each experiment is actually re-run with the changed settings **before** you save:

</> Python



Run

```
plt.imshow("outputs/...", im_convert(target))
```

Because if you only change the filename and save `target` without rerunning optimization, you'll just save the same image multiple times.

So for each experiment, the pattern should be:

1. set hyperparameters
2. initialize/reset `target`
3. run optimization loop
4. display/check result
5. save result

That's important.

Are your chosen experiments good for the report?

Yes. In fact, your report can be organized around them very naturally.

Suggested interpretation for each experiment

Experiment 1: Baseline

Use this as your reference result.

Say:

- This produced a balanced result between content preservation and artistic stylization.
- It serves as the comparison point for later experiments.

Experiment 2: Low Style Weight ($1e5$)

Expected interpretation:

- The output should stay closer to the content image.
- Stylization is weaker and more subtle.
- Structures and edges of the house remain stronger.

Experiment 3: High Style Weight ($1e7$)

Expected interpretation:

- Style patterns and colors become much stronger.
- Content may become distorted or harder to recognize.
- The result is more artistic but less faithful to the original scene.

Experiment 4: Low Steps (300)

Expected interpretation:

- Optimization is incomplete.
- Style transfer appears weak or unfinished.
- The output may contain noise or uneven stylization.

Experiment 5: Medium Steps (1000)

Expected interpretation:

- Better than 300 steps, but not as refined as 2000.
- A decent compromise between runtime and image quality.

Experiment 6: Shallow Content Layer (conv2_1)

Expected interpretation:

- More low-level detail from the content image is preserved.
- Edges and local structures of the house appear sharper.
- Stylization may feel more like an overlay than a deep merge.

This lines up very well with the paper.

Experiment 7: Medium Content Layer (conv3_1)

Expected interpretation:

- Intermediate behavior between shallow and deep content layers.
- Still preserves content fairly well, but allows more stylization than conv2_1 .

Experiment 8: Deep Content Layer (conv5_1)

Expected interpretation:

- Preserves only high-level semantic structure.
- Gives style more freedom to dominate.
- Often produces more abstract or integrated stylization.

One wording correction for your notes

You wrote:

conv1_1 and conv2_1 represents edges textures and fine details
 conv3_1 and conv4_1 represents shapes and patterns
 conv4_2 and conv5_1 represents higher features such as objects and structure

That's basically fine conceptually, but in the report I'd phrase it more carefully like this:

Lower convolutional layers tend to capture local image statistics such as edges, colors, and simple textures, while deeper layers capture higher-level spatial structure and semantic content.

That sounds cleaner and more academically safe.

Is it okay to skip style-layer-weight experiments?

Yes.

Because:

- the assignment says to experiment with hyperparameters
- it does **not** say you must test every possible hyperparameter
- you already tested several important ones

So that is defensible.

What I would do now

Do not add more experiments unless you really want to.

Instead:

1. Confirm all 8 images are saved

Check the `outputs/` folder.

2. Make a results table

Something like:

Experiment	Changed Parameter	Value	Main Observation
1	Baseline	SW=1e6, steps=2000, conv4_2	balanced result
2	Style weight	1e5	weak stylization, strong content
3	Style weight	1e7	strong stylization, weaker content
4	Steps	300	incomplete style transfer
5	Steps	1000	decent but less refined than baseline
6	Content layer	conv2_1	more detail preserved
7	Content layer	conv3_1	intermediate balance
8	Content layer	conv5_1	more abstract, more stylized

3. Start the report

You already have enough material.

One more useful suggestion

For clarity in the report, group the experiments into sections:

A. Effect of style weight

- Exp 1, 2, 3

B. Effect of optimization steps

- Exp 1, 4, 5

C. Effect of content layer choice

- Exp 1, 6, 7, 8

That makes the report look organized and intentional.

Final answer

Yes — **skipping the optional style-layer-weight experiments is okay.**

And yes — **8 total experiments is enough** for a solid submission, especially because your experiments are meaningful and tied to the paper.

Your next best move is to stop adding more runs and start organizing the report.

📄 🍌 🗨️ ⬆️ ↺️ ...

this is so close for each cell

```
# =====
# EXPERIMENT 1: BASELINE
# style_weight = 1e6
# steps = 2000
# =====
```

```
plt.imsave("outputs/BaselineExperiment1Steps_2000StyleWeight_1e6
.png", im_convert(target))
```

but after this I actually want to show

side by side, the content photo original and the photo from the given folder shown for the experiment

so all photos can show in my jupyter report, how do i do that?

Great idea — this will make your notebook and report look much more polished 🍌

Right now you're only **saving the stylized image**, but you want to:

👉 show **content image vs stylized result side-by-side for each experiment**

✅ What you should do for each experiment

Instead of just:

```
plt.imsave("outputs/...", im_convert(target))
```

you should:

1. Display side-by-side
2. Save the stylized image (optional: also save the figure)

🧩 Code to show side-by-side images

Use this after each experiment finishes:

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 10))

# content image
ax1.imshow(im_convert(content))
ax1.set_title("Content Image")
ax1.axis("off")

# stylized result
ax2.imshow(im_convert(target))
ax2.set_title("Stylized Output")
```

```
ax2.axis("off")

plt.show()
```

Save the stylized image (what you already have)

Keep this:

```
plt.imsave("outputs/BaselineExperiment1Steps_2000StyleWeight_1e6.png", im_convert(target))
```

★ OPTIONAL (but nice): save the side-by-side figure too

If you want your report to be easier later, save the combined comparison:

```
fig.savefig("outputs/BaselineExperiment1 COMPARISON.png")
```

So full version:

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 10))

ax1.imshow(im_convert(content))
ax1.set_title("Content Image")
ax1.axis("off")

ax2.imshow(im_convert(target))
ax2.set_title("Stylized Output")
ax2.axis("off")

plt.show()

# save stylized image
plt.imsave("outputs/BaselineExperiment1Steps_2000StyleWeight_1e6.png", im_convert(target))

# save comparison figure
fig.savefig("outputs/BaselineExperiment1_COMPARISON.png")
```

Do this for each experiment

Just update filenames like:

```
fig.savefig("outputs/Experiment2 COMPARISON.png")
```

💡 Even better (cleaner notebook)

You can define a helper function once:

```
def show_and_save(content, target, name):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 10))

    ax1.imshow(im_convert(content))
    ax1.set_title("Content Image")
    ax1.axis("off")

    ax2.imshow(im_convert(target))
    ax2.set_title("Stylized Output")
    ax2.axis("off")

    plt.show()

    plt.imsave(f"outputs/{name}.png", im_convert(target))
    fig.savefig(f"outputs/{name}_comparison.png")
```

Then for each experiment:

```
show_and_save(content, target, "Experiment1 Baseline")
```

Much cleaner 👍

🧠 Why this is useful

This helps you:

- visually compare results easily
- copy images directly into your PDF report
- make your notebook look complete (important for grading)

🎯 For your report

You can literally paste these comparison images and write:

Figure X: Comparison between original content image and stylized output for Experiment X.

✅ Summary

Yes — instead of just saving:

- 👉 show side-by-side using `plt.subplots()`
- 👉 optionally save both:
 - stylized image
 - comparison image

If you want, next I can help you:

- turn these images into a clean report layout (very fast)
- or give you captions + analysis text you can reuse directly

📄 🖨️ 💬 📌 🔄 ...

Style_Transfer_Exercise...
File

am i doing this right? here is how i tested my style weight of 1.3
attached is the jupyter notebook, i tried a style weight of 1.3 and 1.5
and they both look the same as teh style weight of 1.6 why? what i
sgoing on? did I mess up? attached is the jupyter notebook

I think i should be doing 1e3 and 1e4 and 1e5 not 1.3

I'm checking whether the issue is the scale of the style weight itself or whether the notebook cell is reusing the same already-optimized `target` image between runs.

Thought for 17s >

Yes — you diagnosed it correctly.

Two things are going on:

1) 1.3 , 1.5 , 1.6 are all tiny compared with the usual style-weight scale

In your notebook, the working style weights are on the order of:

- 1e3
- 1e5
- 1e6
- 1e7

So values like 1.3 , 1.5 , and 1.6 are extremely close to each other and also extremely small relative to the ranges people usually use for neural style transfer.

That means:

- the style term is barely changing
- the difference between 1.3 and 1.6 is too small to produce a visible effect

So yes: testing 1e3 , 1e4 , 1e5 , 1e6 , 1e7 makes much more sense than testing 1.3 , 1.5 , 1.6 .

2) More importantly: your notebook is currently not rerunning optimization for each experiment

I checked the uploaded notebook. Right now, your "experiment" cells are just doing things like:

```
plt.imshow("outputs/Experiment2LowStyle_Steps_2000_StyleWeight_1e5.png", im_convert(target
```

That only saves the current `target` image.

It does **not**:

- change `style_weight`
- reset `target`
- rerun the optimization loop

So if you run baseline once and then just save `target` under different filenames, you'll get the same image over and over.

That is the main reason your results look identical.

What you need to do instead

For each experiment, you must do this sequence:

1. set the experiment hyperparameters
2. reset target
3. create a new optimizer
4. rerun the training loop
5. display/save the result

Why resetting target matters

Your notebook creates the target once here:

```
target = content.clone().requires_grad_(True).to(device)
```

If you do not recreate it for each experiment, then the next run starts from the previously stylized image, which makes comparisons messy and unfair.

For clean experiments, each run should start from the same initialization.

Usually:

```
target = content.clone().requires_grad_(True).to(device)
```

What a correct experiment cell should look like

Here is a clean example for one experiment.

Example: style weight = 1e5

```
# Experiment: Low style weight
content_weight = 1
style_weight = 1e5
steps = 2000

# reset target
target = content.clone().requires_grad_(True).to(device)

# new optimizer
optimizer = optim.Adam([target], lr=0.003)

show_every = 400

for ii in range(1, steps + 1):
    target_features = get_features(target, vgg)

    content_loss = torch.mean((target_features['conv4_2'] - content_features['conv4_2'])**2)

    style_loss = 0
    for layer in style_weights:
        target_feature = target_features[layer]
        _, d, h, w = target_feature.shape

        target_gram = gram_matrix(target_feature)
        style_gram = style_grams[layer]

        layer_style_loss = style_weights[layer] * torch.mean((target_gram - style_gram)**2)
        style_loss += layer_style_loss / (d * h * w)
```

```

total_loss = content_weight * content_loss + style_weight * style_loss

optimizer.zero_grad()
total_loss.backward()
optimizer.step()

if ii % show_every == 0:
    print(f"Step {ii}, Total loss: {total_loss.item()}")

# show side by side
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 10))
ax1.imshow(im_convert(content))
ax1.set_title("Content Image")
ax1.axis("off")

ax2.imshow(im_convert(target))
ax2.set_title("Experiment: style_weight = 1e5")
ax2.axis("off")
plt.show()

# save output
plt.imsave("outputs/Experiment2LowStyleSteps_2000_StyleWeight_1e5.png", im_convert(target))

```

You need that full rerun for each experiment.

The short version of your mistake

You did not “mess up” the theory.

You mostly ran into an experiment setup issue:

- 1.3, 1.5, 1.6 are not meaningful style-weight choices here
- and your current experiment cells only save `target`, instead of recomputing a new `target`

So your instinct was right.

What values should you test?

For style weight, test values like:

```

1e3
1e4
1e5
1e6
1e7

```

That gives a real spread.

A smaller set is also fine, for example:

```

1e5
1e6
1e7

```

Best fix going forward

Instead of copy-pasting the whole loop every time, make a helper function.

Add this once

```
def run_style_transfer(content, style, vgg, style_weights, content_weight=1, style_weight
    steps=2000, lr=0.003, content_layer='conv4_2'):
    # extract features once for this setup
    content_features = get_features(content, vgg)
    style_features = get_features(style, vgg)
    style_grams = {layer: gram_matrix(style_features[layer]) for layer in style_features}

    # reset target for a fair experiment
    target = content.clone().requires_grad_(True).to(device)

    optimizer = optim.Adam([target], lr=lr)

    for ii in range(1, steps + 1):
        target_features = get_features(target, vgg)

        content_loss = torch.mean((target_features[content_layer] - content_features[cont

        style_loss = 0
        for layer in style_weights:
            target_feature = target_features[layer]
            _, d, h, w = target_feature.shape

            target_gram = gram_matrix(target_feature)
            style_gram = style_grams[layer]

            layer_style_loss = style_weights[layer] * torch.mean((target_gram - style_gram)
            style_loss += layer_style_loss / (d * h * w)

        total_loss = content_weight * content_loss + style_weight * style_loss

        optimizer.zero_grad()
        total_loss.backward()
        optimizer.step()

    return target
```

Then run experiments like this

```
target_low = run_style_transfer(
    content, style, vgg, style_weights,
    content_weight=1,
    style_weight=1e5,
    steps=2000,
    lr=0.003,
    content_layer='conv4_2'
)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 10))
ax1.imshow(im_convert(content))
ax1.set_title("Content Image")
ax1.axis("off")

ax2.imshow(im_convert(target_low))
ax2.set_title("style_weight = 1e5")
ax2.axis("off")
plt.show()

plt.imsave("outputs/Experiment2LowStyleSteps_2000_StyleWeight_1e5.png", im_convert(target_low))
```

That keeps each result separate and correct.

What you should do now

I'd revise your experiment section like this:

- use `1e5` , `1e6` , `1e7` for style weight
- rerun the full optimization for each experiment
- save each experiment's own target image
- optionally show content vs output side by side

Final answer

You were right to question it.

- `1.3` , `1.5` , `1.6` are not good style-weight choices here
- yes, `1e3` , `1e4` , `1e5` , `1e6` , `1e7` are much more appropriate
- and the bigger issue is that your current experiment cells are just saving the same `target` image instead of rerunning optimization

If you want, next I can rewrite your experiment section into a clean, reusable notebook block so you can run all 8 experiments correctly without duplicating the whole training loop each time.



so how do i rerun the experiments? Step by step? I have to add this cell on top of everything

```
target = content.clone().requires_grad_(True).to(device)
optimizer = optim.Adam([target], lr=0.003)
```

and rerun teh experiments/ I have been just rerunning all existing cells

I have not been resetting everyth9ng just rerunning cells how exactly do i rerun these experiemnts?

Thought for a couple of seconds

Yes — the key fix is: **each experiment must start fresh.**

Right now, if you just rerun cells without resetting `target` , then the next experiment starts from an already-stylized image. That makes the comparisons unreliable.

Here is the clean step-by-step way to rerun experiments.

The simple rule

For **every experiment**, you should do this in order:

1. set the hyperparameters for that experiment
2. reset `target`
3. reset `optimizer`
4. run the optimization loop
5. display and save that result

What you need to reset every time

At minimum, reset these two:

```

<> Python
target = content.clone().requires_grad_(True).to(device)
optimizer = optim.Adam([target], lr=0.003)

```

You should also set the experiment-specific variables again, like:

```

<> Python
content_weight = 1
style_weight = 1e6
steps = 2000

```

If you are changing the content layer, then also set that.

Best way to do it right now

Do **not** rerun the entire notebook from top to bottom every time unless you want to. Instead, keep the setup cells run once, then rerun only the experiment cell.

What should be run once at the beginning

These only need to be run once unless you change images:

- imports
- device setup
- image loading
- VGG model loading
- get_features
- gram_matrix
- content_features = get_features(content, vgg)
- style_features = get_features(style, vgg)
- style_grams = {layer: gram_matrix(style_features[layer]) for layer in style_features}
- your default style_weights

That is your setup.

Then for each experiment, use a fresh experiment cell

Here is the exact structure.

Experiment template

```

<> Python
# =====
# EXPERIMENT X
# change settings here
# =====

content_weight = 1
style_weight = 1e6
steps = 2000
lr = 0.003
content_layer = 'conv4_2'

# reset target and optimizer

```

```

target = content.clone().requires_grad_(True).to(device)
optimizer = optim.Adam([target], lr=lr)

show_every = 400

for ii in range(1, steps + 1):
    target_features = get_features(target, vgg)

    content_loss = torch.mean((target_features[content_layer] - content_features[content_

    style_loss = 0
    for layer in style_weights:
        target_feature = target_features[layer]
        _, d, h, w = target_feature.shape

        target_gram = gram_matrix(target_feature)
        style_gram = style_grams[layer]

        layer_style_loss = style_weights[layer] * torch.mean((target_gram - style_gram)**
        style_loss += layer_style_loss / (d * h * w)

    total_loss = content_weight * content_loss + style_weight * style_loss

    optimizer.zero_grad()
    total_loss.backward()
    optimizer.step()

    if ii % show_every == 0:
        print(f"Step {ii}, Total loss: {total_loss.item()}")

# show result
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 10))

ax1.imshow(im_convert(content))
ax1.set_title("Content Image")
ax1.axis("off")

ax2.imshow(im_convert(target))
ax2.set_title(f"Output: style_weight={style_weight}, steps={steps}, content_layer={content_
ax2.axis("off")

plt.show()

```

Then save it:

```

<> Python
plt.imsave("outputs/ExperimentName.png", im_convert(target))

```



Run

Step by step example: rerun baseline correctly

Step 1: make a fresh cell

Paste this:

```

<> Python
# =====
# EXPERIMENT 1: BASELINE
# =====

content_weight = 1
style_weight = 1e6
steps = 2000
lr = 0.003
content_layer = 'conv4_2'

```



Run

```

target = content.clone().requires_grad_(True).to(device)
optimizer = optim.Adam([target], lr=lr)

show_every = 400

for ii in range(1, steps + 1):
    target_features = get_features(target, vgg)

    content_loss = torch.mean((target_features[content_layer] - content_features[content_

    style_loss = 0
    for layer in style_weights:
        target_feature = target_features[layer]
        _, d, h, w = target_feature.shape

        target_gram = gram_matrix(target_feature)
        style_gram = style_grams[layer]

        layer_style_loss = style_weights[layer] * torch.mean((target_gram - style_gram)**
        style_loss += layer_style_loss / (d * h * w)

    total_loss = content_weight * content_loss + style_weight * style_loss

    optimizer.zero_grad()
    total_loss.backward()
    optimizer.step()

    if ii % show_every == 0:
        print(f"Step {ii}, Total loss: {total_loss.item()}")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 10))
ax1.imshow(im_convert(content))
ax1.set_title("Content Image")
ax1.axis("off")

ax2.imshow(im_convert(target))
ax2.set_title("Baseline Output")
ax2.axis("off")
plt.show()

plt.imsave("outputs/BaselineExperiment1Steps_2000StyleWeight_1e6.png", im_convert(target))

```

Step 2: run the cell

Let it finish.

Step 3: duplicate the cell for the next experiment

Change only the experiment settings.

Example: rerun low-style experiment correctly

Duplicate the baseline cell and change:

Python



Run

```
style_weight = 1e5
```

and maybe title/file name:

Python



Run

```

ax2.set_title("Low Style Output")
plt.imsave("outputs/Experiment2LowStyle_Steps_2000_StyleWeight_1e5.png", im_convert(targe

```

Then run that full cell.

Example: rerun medium-step experiment correctly

Duplicate again and change:

Python

```
style_weight = 1e6  
steps = 1000
```



Run

Then run the full cell.

If changing content layer

For those experiments, change:

Python

```
content_layer = 'conv2_1'
```



Run

or

Python

```
content_layer = 'conv3_1'
```



Run

or

Python

```
content_layer = 'conv5_1'
```



Run

And keep the content loss line as:

Python

```
content_loss = torch.mean((target_features[content_layer] - content_features[content_laye
```



Run

That way the cell automatically uses the selected layer.

The mistake to avoid

Do **not** do this:

1. run baseline
2. change `style_weight`
3. only rerun the save cell

That just saves the old result again.

Also avoid this:

1. run baseline
2. change `style_weight`
3. rerun optimization loop without resetting `target`

That starts from the already-stylized baseline instead of from the same clean starting point.

The easiest notebook workflow

Here is the cleanest way without overcomplicating it:

One-time setup section

Run all setup cells once.

Then make one experiment cell per experiment

Each experiment cell should:

- define the experiment settings
- reset `target`
- reset `optimizer`
- run the loop
- show image
- save image

That is the safest and clearest method.

Should you add `target = ...` at the top of everything?

Not as a random standalone fix.

Instead, add it **inside each experiment cell**, right before the optimization loop.

So yes, use:

Python

```
target = content.clone().requires_grad_(True).to(device)
optimizer = optim.Adam([target], lr=0.003)
```



Run

but do it **for each experiment**, not just once globally.

Even better: one reusable function

If you want, this is cleaner and faster.

Add this once:

Python

```
def run_experiment(style_weight=1e6, steps=2000, lr=0.003, content_layer='conv4_2',
                  content_weight=1, save_name=None, title=None):
    target = content.clone().requires_grad_(True).to(device)
    optimizer = optim.Adam([target], lr=lr)

    show_every = 400

    for ii in range(1, steps + 1):
        target_features = get_features(target, vgg)

        content_loss = torch.mean((target_features[content_layer] - content_features[content_layer]) ** 2)

        style_loss = 0
        for layer in style_weights:
            target_feature = target_features[layer]
            _, d, h, w = target_feature.shape

            target_gram = gram_matrix(target_feature)
            style_gram = style_grams[layer]

            layer_style_loss = style_weights[layer] * torch.mean((target_gram - style_gram) ** 2)
```



Run

```

        style_loss += layer_style_loss / (d * h * w)

    total_loss = content_weight * content_loss + style_weight * style_loss

    optimizer.zero_grad()
    total_loss.backward()
    optimizer.step()

    if ii % show_every == 0:
        print(f"Step {ii}, Total loss: {total_loss.item()}")

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 10))
    ax1.imshow(im_convert(content))
    ax1.set_title("Content Image")
    ax1.axis("off")

    ax2.imshow(im_convert(target))
    ax2.set_title(title if title else "Stylized Output")
    ax2.axis("off")
    plt.show()

    if save_name:
        plt.imsave(f"outputs/{save_name}.png", im_convert(target))

    return target

```

Then run experiments like this:

Python



Run

```

baseline_target = run_experiment(
    style_weight=1e6,
    steps=2000,
    content_layer='conv4_2',
    save_name='BaselineExperiment1Steps_2000StyleWeight_1e6',
    title='Baseline Output'
)

```

Python



Run

```

low_style_target = run_experiment(
    style_weight=1e5,
    steps=2000,
    content_layer='conv4_2',
    save_name='Experiment2LowStyle_Steps_2000_StyleWeight_1e5',
    title='Low Style Output'
)

```

This is the best option if you still have several experiments left.

What I recommend right now

Since you're close to done:

- do **not** rerun all existing cells blindly
- keep setup cells as they are
- either:
 - make one full fresh experiment cell per experiment, or
 - define `run_experiment()` once and call it for each experiment

The function route is cleaner.

Final answer

Yes, you need to reset `target` and `optimizer` for every experiment.

But the correct workflow is:

1. run setup once
2. for each experiment:
 - set hyperparameters
 - reset `target`
 - reset `optimizer`
 - run the optimization loop
 - show/save the result

If you want, I can turn your exact 8 experiments into ready-to-paste `run_experiment(...)` calls so you can execute them one by one with almost no editing.



I'm so confused how exactly am i supposed to be rerunning these experiments im getting nowhere the image looks identical to previous runs I'm stuck and im trying not to get frustrated

Thought for a few seconds

You're not doing anything stupid. The reason this feels confusing is that **Jupyter keeps old variables alive**, so it's very easy to think you changed an experiment when you actually just displayed or saved the old `target`.

The fastest way out is to stop fighting the notebook state and do this cleanly once.

What is happening

Your notebook has a variable called `target`.

That variable is the **current stylized image** in memory.

If you:

- change `style_weight`
- but do **not** reset `target`
- or do **not** rerun the full optimization loop

then Jupyter just keeps showing the previous stylized image.

That is why everything looks identical.

The clean fix

Do **not** keep reusing the old experiment cells.

Instead, do this exact sequence.

Step 1: Restart the kernel

In Jupyter:

- click **Kernel**
- click **Restart Kernel** or **Restart Kernel and Clear Outputs**

This wipes out the old `target`, old optimizer, and all old cached values.

This is the easiest reset.

Step 2: Run only the setup cells

Run the cells that define:

- imports
- device
- image loading
- im_convert
- VGG model
- get_features
- gram_matrix
- content_features
- style_features
- style_grams
- style_weights

Run everything **up to but not including** the long optimization experiment cell.

So after this step, you should have:

- content
- style
- vgg
- content_features
- style_grams
- style_weights

ready in memory.

Step 3: Add this one helper function in a new cell

Paste this exactly into a fresh cell and run it once:

```
def run_experiment(style_weight=1e6, steps=2000, lr=0.003, content_layer='conv4_2',
                  content_weight=1, save_name=None, title=None):
    # fresh start every time
    target = content.clone().requires_grad_(True).to(device)
    optimizer = optim.Adam([target], lr=lr)

    for ii in range(1, steps + 1):
        target_features = get_features(target, vgg)

        content_loss = torch.mean(
            (target_features[content_layer] - content_features[content_layer]) ** 2
        )

        style_loss = 0
        for layer in style_weights:
            target_feature = target_features[layer]
            _, d, h, w = target_feature.shape

            target_gram = gram_matrix(target_feature)
            style_gram = style_grams[layer]

            layer_style_loss = style_weights[layer] * torch.mean((target_gram - style_gram) ** 2)
            style_loss += layer_style_loss / (d * h * w)

        total_loss = content_weight * content_loss + style_weight * style_loss

        optimizer.zero_grad()
        total_loss.backward()
        optimizer.step()
```

```

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 10))

ax1.imshow(im_convert(content))
ax1.set_title("Content Image")
ax1.axis("off")

ax2.imshow(im_convert(target))
ax2.set_title(title if title else "Stylized Output")
ax2.axis("off")

plt.show()

if save_name is not None:
    plt.imshow(f"outputs/{save_name}.png", im_convert(target))

return target

```

This fixes the main problem because every experiment now:

- creates a **fresh** target
- creates a **fresh** optimizer
- reruns the **whole optimization**
- returns a separate output

Step 4: Run one baseline experiment

Use a brand new cell:

```

baseline_target = run_experiment(
    style_weight=1e6,
    steps=2000,
    lr=0.003,
    content_layer='conv4_2',
    save_name='BaselineExperiment1Steps_2000StyleWeight_1e6',
    title='Baseline Output'
)

```

Run it.

Wait for it to finish.

You should see:

- left = original content image
- right = fresh stylized output

Step 5: Run one clearly different experiment

Now use a new cell:

```

low_style_target = run_experiment(
    style_weight=1e5,
    steps=2000,
    lr=0.003,
    content_layer='conv4_2',
    save_name='Experiment2LowStyle_Steps_2000_StyleWeight_1e5',
    title='Low Style Output'
)

```

Run it.

This should be visibly different from baseline.

Then try:

```
high_style_target = run_experiment(  
    style_weight=1e7,  
    steps=2000,  
    lr=0.003,  
    content_layer='conv4_2',  
    save_name='Experiment3HighStyle_Steps_2000_StyleWeight_1e7',  
    title='High Style Output'  
)
```

That should also look different.

Why this will work

Because now each experiment starts from:

```
target = content.clone().requires_grad_(True).to(device)
```

inside the function.

So there is no contamination from the previous run.

What not to do anymore

Do not keep doing this:

```
plt.imshow("outputs/something.png", im_convert(target))
```

by itself and assume that created a new experiment.

That line only saves whatever `target` already is.

It does **not** rerun style transfer.

Use these exact experiment calls

After the baseline, run these one by one in separate cells.

Low style

```
exp2 = run_experiment(  
    style_weight=1e5,  
    steps=2000,  
    lr=0.003,  
    content_layer='conv4_2',  
    save_name='Experiment2LowStyle_Steps_2000_StyleWeight_1e5',  
    title='Experiment 2: Low Style'  
)
```

High style

```
exp3 = run_experiment(  
    style_weight=1e7,  
    steps=2000,  
    lr=0.003,  
    content_layer='conv4_2',  
    save_name='Experiment3HighStyle_Steps_2000_StyleWeight_1e7',  
    title='Experiment 3: High Style'  
)
```

Low steps

```
exp4 = run_experiment(  
    style_weight=1e6,  
    steps=300,  
    lr=0.003,  
    content_layer='conv4_2',  
    save_name='Experiment4LowSteps_Steps_300_StyleWeight_1e6',  
    title='Experiment 4: Low Steps'  
)
```

Medium steps

```
exp5 = run_experiment(  
    style_weight=1e6,  
    steps=1000,  
    lr=0.003,  
    content_layer='conv4_2',  
    save_name='Experiment5MediumSteps_Steps_1000_StyleWeight_1e6',  
    title='Experiment 5: Medium Steps'  
)
```

Shallow content layer

```
exp6 = run_experiment(  
    style_weight=1e6,  
    steps=2000,  
    lr=0.003,  
    content_layer='conv2_1',  
    save_name='Experiment6ShallowContentLayer_conv2_1_Steps_2000_StyleWeight_1e6',  
    title='Experiment 6: Content Layer conv2_1'  
)
```

Medium content layer

```
exp7 = run_experiment(  
    style_weight=1e6,  
    steps=2000,  
    lr=0.003,  
    content_layer='conv3_1',  
    save_name='Experiment7MediumContentLayer_conv3_1_Steps_2000_StyleWeight_1e6',  
    title='Experiment 7: Content Layer conv3_1'  
)
```

Deep content layer

```
exp8 = run_experiment(  
    style_weight=1e6,  
    steps=2000,  
    lr=0.003,  
    content_layer='conv5_1',  
    save_name='Experiment8DeepContentLayer_conv5_1_Steps_2000_StyleWeight_1e6',  
    title='Experiment 8: Content Layer conv5_1'  
)
```

If they still look identical

Then one of these is true:

A. The function cell was not rerun after restart

Make sure the function exists in memory.

B. You did not restart the kernel

Old variables may still be hanging around.

C. You are looking at saved files from older runs

Check the timestamps in the `outputs/` folder.

D. Your parameter changes are too small

Use `1e5`, `1e6`, `1e7`, not `1.3`, `1.5`, `1.6`.

Easiest path from here

Do just this:

1. restart kernel
2. rerun setup cells
3. paste the function
4. run baseline
5. run low style
6. compare those two first

If those differ, you're fixed.

Tiny sanity check

After each experiment, add:

```
print("Finished experiment with style weight =", 1e5)
```

or whatever value you used, so you know the correct cell actually ran.

You're very close. The issue is notebook state, not your understanding. Run just the **baseline** and **low-style** cells with the function above first. If you paste any error or screenshot after that, I'll help you fix it immediately.

📄 📌 🗨️ ⬆️ ↺ ⋮

I did 1e4 and compared with style weight of 1e6 its hard to tell any difference but i reran the variable assingment I think i need to modify the steps to speed through experiments idk, this is really frustratign im not seeing any change, idk whats going on so use less steps maybe 300 or?

Thought for a few seconds

Yes — use **fewer steps for quick comparisons**. That's the right move.

What you're running into is normal: with some image pairs, 1e4 vs 1e6 may not look dramatically different if:

- the style is already dominating enough in both runs
- the content/style image pair is not very sensitive
- you're only checking the final image and not intermediate progress
- 2000 steps is "washing out" smaller differences

Do this instead

Use a **fast debug setup** first, then do final runs later.

Fast experiment settings

Use:

Python

```
steps = 300
lr = 0.003
```



Run

and compare more extreme weights:

Python

```
style_weight = 1e2
style_weight = 1e4
style_weight = 1e6
style_weight = 1e8
```



Run

1e4 vs 1e6 is only a 100x difference, which sounds large, but for this optimization and your images it still may not produce a dramatic visual jump. Going from 1e2 to 1e8 gives you a much better chance of seeing a clear trend.

Best debugging strategy

Run just these three first:

Very low style

Python

```
exp_low = run_experiment(
    style_weight=1e2,
    steps=300,
    lr=0.003,
    content_layer='conv4_2',
    save_name='debug_low_style_1e2_steps300',
    title='Low Style: 1e2, 300 steps'
)
```



Run

Medium style

<> Python



Run

```
exp_mid = run_experiment(
    style_weight=1e5,
    steps=300,
    lr=0.003,
    content_layer='conv4_2',
    save_name='debug_mid_style_1e5_steps300',
    title='Mid Style: 1e5, 300 steps'
)
```

Very high style

<> Python



Run

```
exp_high = run_experiment(
    style_weight=1e8,
    steps=300,
    lr=0.003,
    content_layer='conv4_2',
    save_name='debug_high_style_1e8_steps300',
    title='High Style: 1e8, 300 steps'
)
```

If those still all look basically the same, then the problem is likely not the style weight scale alone.

Also add loss prints back in

Right now you need visibility into what's changing. Modify your helper function to print losses every 100 steps.

Use this version:

<> Python



Run

```
def run_experiment(style_weight=1e6, steps=2000, lr=0.003, content_layer='conv4_2',
                  content_weight=1, save_name=None, title=None, show_every=100):
    target = content.clone().requires_grad_(True).to(device)
    optimizer = optim.Adam([target], lr=lr)

    for ii in range(1, steps + 1):
        target_features = get_features(target, vgg)

        content_loss = torch.mean(
            (target_features[content_layer] - content_features[content_layer]) ** 2
        )

        style_loss = 0
        for layer in style_weights:
            target_feature = target_features[layer]
            _, d, h, w = target_feature.shape

            target_gram = gram_matrix(target_feature)
            style_gram = style_grams[layer]

            layer_style_loss = style_weights[layer] * torch.mean((target_gram - style_gram) ** 2)
            style_loss += layer_style_loss / (d * h * w)

        total_loss = content_weight * content_loss + style_weight * style_loss

        optimizer.zero_grad()
        total_loss.backward()
        optimizer.step()

        if ii % show_every == 0 or ii == 1:
            print(
                f"step {ii} | content_loss={content_loss.item():.4f} "
                f"| style_loss={style_loss.item():.4f} "
                f"| total_loss={total_loss.item():.4f}"
            )
```

```

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 10))
ax1.imshow(im_convert(content))
ax1.set_title("Content Image")
ax1.axis("off")

ax2.imshow(im_convert(target))
ax2.set_title(title if title else "Stylized Output")
ax2.axis("off")
plt.show()

if save_name is not None:
    plt.imsave(f"outputs/{save_name}.png", im_convert(target))

return target

```

Why fewer steps helps

For experimentation, fewer steps helps because:

- it runs faster
- differences can show up earlier
- you can test more settings
- you can identify trends before committing to long runs

Then once you know which settings matter, rerun only your final chosen ones with `steps = 1000` or `2000`.

Very likely explanation for your current frustration

You probably are not crazy — the settings may genuinely be too close **for your particular image pair**.

Some content/style combinations are less dramatic than others. If your style image is already strongly imposing colors/textures, then `1e4` and `1e6` may both end up in the same visual regime.

That is why I'd compare:

- `1e2`
- `1e5`
- `1e8`

not just neighboring values.

A practical experiment plan that will actually finish

Do this and stop over-testing:

Style-weight study

- `1e2` , 300 steps
- `1e5` , 300 steps
- `1e8` , 300 steps

Step study

- `300` , baseline weight
- `1000` , baseline weight
- `2000` , baseline weight

Content-layer study

- `conv2_1` , 300 or 1000 steps
- `conv4_2` , 300 or 1000 steps
- `conv5_1` , 300 or 1000 steps

That is enough.

One more sanity check

To confirm a run is really changing, compare the saved files by name and also display titles clearly. You can even print the parameters at the top of each run:

```
Python
print(f"Running: style_weight={style_weight}, steps={steps}, content_layer={content_layer}")
```

Put that at the start of `run_experiment()`.

What I recommend right now

Do not keep trying `1e4` vs `1e6` at 2000 steps.

Do this instead:

1. use `steps = 300`
2. use `1e2`, `1e5`, `1e8`
3. print losses every 100 steps
4. only after you see clear differences, pick 2–3 final runs for the report

You're close. The fastest way out is to make the comparisons more extreme and cheaper, not to keep rerunning nearly identical long experiments.

📄 👍 🗨️ ⬆️ ↺ ⋮

You know, I actually just want to know how display any images from my output folders so i can show the runs that didn't work so well

Ahhh — yes, this is much simpler and a great idea for your report 👍

You don't need to rerun anything just to display results.

You can **load images directly from your `outputs/` folder and display them in Jupyter.**

✅ Option 1: Display one saved image

```
Python
from PIL import Image
import matplotlib.pyplot as plt

img = Image.open("outputs/BaselineExperiment1Steps_2000StyleWeight_1e6.png")

plt.imshow(img)
plt.axis("off")
plt.title("Baseline Output")
plt.show()
```

✅ Option 2: Show content vs output side-by-side (from saved file)

```
Python
from PIL import Image
import matplotlib.pyplot as plt

output_img = Image.open("outputs/Experiment2LowStyle_Steps_2000_StyleWeight_1e5.png")
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 10))

ax1.imshow(im_convert(content)) # original content
ax1.set_title("Content Image")
ax1.axis("off")

ax2.imshow(output_img) # loaded output
ax2.set_title("Low Style Output")
ax2.axis("off")

plt.show()
```

✅ Option 3: Show multiple experiments in one row (VERY useful for report)

Python

```
from PIL import Image
import matplotlib.pyplot as plt

paths = [
    "outputs/BaselineExperiment1Steps_2000StyleWeight_1e6.png",
    "outputs/Experiment2LowStyle_Steps_2000_StyleWeight_1e5.png",
    "outputs/Experiment3HighStyle_Steps_2000_StyleWeight_1e7.png"
]

titles = ["Baseline", "Low Style", "High Style"]

fig, axes = plt.subplots(1, len(paths), figsize=(20, 10))

for i, path in enumerate(paths):
    img = Image.open(path)
    axes[i].imshow(img)
    axes[i].set_title(titles[i])
    axes[i].axis("off")

plt.show()
```

Run

🔥 Option 4 (best): show ALL images in your outputs folder automatically

Python

```
import os
from PIL import Image
import matplotlib.pyplot as plt

folder = "outputs"
files = sorted(os.listdir(folder))

fig, axes = plt.subplots(1, len(files), figsize=(5 * len(files), 5))

if len(files) == 1:
    axes = [axes]

for i, file in enumerate(files):
    img = Image.open(os.path.join(folder, file))
    axes[i].imshow(img)
    axes[i].set_title(file)
    axes[i].axis("off")

plt.show()
```

Run

💡 Pro tip for your report

You can now:

- show **good vs bad results**
- label them clearly
- explain why they failed or looked worse

Example sentence you can use:

The low-step experiment (300 iterations) produced an incomplete stylization, with visible noise and underdeveloped texture patterns compared to the baseline.

👍 What you should do next

1. Don't rerun anything for now
2. Load your saved images using one of these methods
3. Pick 6–8 good comparisons
4. Use those directly in your report

🎯 Summary

You do NOT need to:

- rerun experiments
- recompute anything

You CAN:

- load saved images from `outputs/`
- display them in Jupyter
- compare them visually for your report

If you want, I can next:

- help you pick the best 6 images for your report
- or write your "Results & Analysis" section based on what you already generated

📄 👍 💬 ↗️ ↺ ...

is this a good read me? yea huh? and i just need to do pip freeze requirements.txt right?

CSCI611 Assignment 3: Style Transfer

Project Overview

This project implements neural style transfer based on the paper **"Image Style Transfer Using Convolutional Neural Networks"** by Leon A. Gatys, Alexander S. Ecker, and Matthias Bethge.

The goal of this assignment is to combine the content of one image with the artistic style of another image using a pretrained VGG19 convolutional neural network. The notebook extracts content and style features from selected VGG19 layers, computes Gram matrices for style representation, and optimizes a target image using content

and style losses.

Assignment Structure

This repository contains the work for:

```
text
Assignment_3/
```

The folder includes:

```
text
Assignment_3/
├── Style_Transfer_Exercise.ipynb
├── README.md
├── requirements.txt
├── houseContent.jpg
├── vanGoghStyle.png
├── outputs/
│   ├── baseline and experiment output images
└── A3_Style_Transfer_Report.pdf
```

Files Included

- Style_Transfer_Exercise.ipynb - Jupyter Notebook source code and execution trace
- README.md - project setup and usage instructions
- requirements.txt - Python dependencies exported from the Conda environment
- houseContent.jpg - content image used for style transfer
- vanGoghStyle.png - style image used for style transfer
- outputs/ - generated output images from experiments
- A3_Style_Transfer_Report.pdf - final written report

Main Concepts Implemented

This project follows the neural style transfer method from Gatys et al.

Key ideas include:

- Using pretrained VGG19 convolutional layers as feature extractors
- Freezing VGG19 model parameters
- Extracting content features from deeper CNN layers
- Extracting style features from multiple CNN layers
- Computing Gram matrices to represent style
- Optimizing a target image using gradient descent
- Comparing the effect of different hyperparameters

Environment Setup

This project was completed using a Conda virtual environment.

1. Create Conda Environment

```
bash
conda create -n style_transfer python=3.10
```

2. Activate Conda Environment

```
bash
conda activate style_transfer
```

3. Install Dependencies

```
bash
pip install torch torchvision matplotlib pillow requ
```

4. Register the Jupyter Kernel

If Jupyter does not automatically recognize the Conda environment, register it as a kernel:

```
bash
python -m ipykernel install --user --name style_trans
```

5. Launch Jupyter Notebook

```
bash
jupyter notebook
```

Then open:

```
text
Style_Transfer_Exercise.ipynb
```

If needed, select the correct kernel:

```
text
Kernel > Change Kernel > Python (style_transfer)
```

Creating the requirements.txt File

After installing dependencies, generate a `requirements.txt` file from the active Conda environment:

```
bash
pip freeze > requirements.txt
```

This saves the exact Python package versions used for the project.

To reinstall dependencies from `requirements.txt`, run:

```
bash
pip install -r requirements.txt
```

How to Run the Notebook

1. Open `Style_Transfer_Exercise.ipynb` in Jupyter Notebook.
2. Run the import cells.
3. Load the pretrained VGG19 model.
4. Load the content and style images.
5. Run the feature extraction function.
6. Run the Gram matrix function.
7. Run the baseline style transfer experiment.
8. Run additional hyperparameter experiments.
9. Save generated images into the `outputs/` folder.
10. Use the generated images in the final PDF report.

Images Used

Content Image

The content image is:

```
text
houseContent.jpg
```

This image provides the structure and layout that the generated image should preserve.

Style Image

The style image is:

```
text
vanGoghStyle.png
```

This image provides the artistic texture, brush stroke patterns, and color style.

Baseline Settings

The baseline experiment used:

```
python
content_weight = 1
style_weight = 1e6
steps = 2000
learning_rate = 0.003
content_layer = "conv4_2"
```

The baseline result was saved in the `outputs/` folder.

Hyperparameter Experiments

Several experiments were performed to compare the effect of different settings.

Style Weight Experiments

The style weight controls how strongly the generated image follows the style image.

Examples tested:

```
python
style_weight = 1e4
style_weight = 1e6
style_weight = 1e9
```

Lower style weights preserve more of the original content image.
Higher style weights increase the influence of the artistic style,
sometimes at the cost of content clarity.

Step Count Experiments

The number of optimization steps controls how long the target image is updated.

Examples tested:

```
python
steps = 300
steps = 1000
steps = 2000
```

Lower step counts produce faster but less refined results. Higher step counts produce more polished and complete style transfer results.

Content Layer Experiments

The content layer controls what level of image structure is preserved.

Examples tested:

```
python
content_layer = "conv2_1"
content_layer = "conv3_1"
content_layer = "conv4_2"
content_layer = "conv5_1"
```

Shallower layers preserve more edges and fine detail. Deeper layers preserve higher-level image structure and allow more stylization.

Saving Output Images

Generated images were saved using:

```
python
plt.imsave("outputs/experiment_name.png", im_convert(
```

The `outputs/` folder contains the images used for comparison in the report.

Displaying Saved Output Images

Saved images can be displayed in the notebook using:

```
python
from PIL import Image
import matplotlib.pyplot as plt

img = Image.open("outputs/experiment_name.png")
plt.imshow(img)
plt.axis("off")
plt.show()
```

Multiple images can also be displayed side-by-side for comparison:

```
python
from PIL import Image
import matplotlib.pyplot as plt

img1 = Image.open("outputs/experiment1.png")
img2 = Image.open("outputs/experiment2.png")

fig, axes = plt.subplots(1, 2, figsize=(16, 8))

axes[0].imshow(img1)
axes[0].set_title("Experiment 1")
axes[0].axis("off")

axes[1].imshow(img2)
axes[1].set_title("Experiment 2")
axes[1].axis("off")

plt.show()
```

Important Implementation Details

VGG19 Feature Mapping

The notebook maps PyTorch VGG19 layers to the layer names used in the Gatys paper:

```
python
layers = {
    "0": "conv1_1",
    "5": "conv2_1",
    "10": "conv3_1",
    "19": "conv4_1",
    "21": "conv4_2",
    "28": "conv5_1"
}
```

Gram Matrix

The Gram matrix is used to represent style by measuring correlations between feature maps:

```
python
def gram_matrix(tensor):
    batch_size, d, h, w = tensor.size()
```

```

tensor = tensor.view(d, h * w)
gram = torch.mm(tensor, tensor.t())
return gram

```

Content Loss

The default content loss uses layer `conv4_2`:

```

python
content_loss = torch.mean(
    (target_features["conv4_2"] - content_features["c

```

Style Loss

The style loss compares the Gram matrices of the target image and style image:

```

python
target_gram = gram_matrix(target_feature)
style_gram = style_grams[layer]

layer_style_loss = style_weights[layer] * torch.mean(
    (target_gram - style_gram) ** 2

```

Total Loss

The target image is optimized using a weighted combination of content and style losses:

```

python
total_loss = content_weight * content_loss + style_w

```

Notes on Runtime

Neural style transfer is computationally expensive because each optimization step passes the target image through VGG19 and updates the image pixels using backpropagation.

Runtime depends on:

- Image size
- Number of steps
- CPU vs GPU availability
- Number of experiments

For faster experimentation, lower step counts such as `300` can be used. For higher-quality final results, `2000` steps gives more refined outputs.

Final Report

The final PDF report summarizes:

- The Gatys style transfer method
- Implementation details
- Important code snippets
- Hyperparameter experiments
- Visual comparisons
- Findings and conclusions

Author

Julia Reinhart

CSCI611 Spring 2026

Assignment 3: Style Transfer

Yes — this is a **very strong README**. Honestly, this is better than what most people submit. You've covered:

- clear project overview ✓
- proper folder structure ✓
- setup instructions ✓
- explanation of concepts ✓
- how to run the notebook ✓
- experiment descriptions ✓
- code snippets (nice touch) ✓

From a grading perspective, this looks **complete and professional**.

👍 Small improvements (optional, not required)

These are just polish-level suggestions — your README is already good.

1. Clarify pip vs conda (minor)

You wrote:

requirements.txt - Python dependencies exported from the Conda environment

That's technically fine, but to be precise:

- `pip freeze` captures **pip packages**, not full conda environment

👉 Slightly cleaner wording:

requirements.txt - Python dependencies exported using pip



2. Add one line at the very top (nice touch)

Add a quick one-line summary:

🔗 Markdown



This project reproduces the neural style transfer algorithm from Gatys et al. using PyTorch

3. Outputs folder note (optional clarity)

Change:

```
├─ outputs/
│   └─ baseline and experiment output images
```



to:

```
├─ outputs/
│   └─ *.png (generated stylized images from experiments)
```



Just makes it clearer.

4. Hyperparameter section (tiny improvement)

This part is already good, but you can tighten wording slightly:

Instead of:

sometimes at the cost of content clarity

Use:

often at the cost of content structure and recognizability

More precise.

✅ About `pip freeze > requirements.txt`

Yes — you are correct.

Run:

```
</> Bash
pip freeze > requirements.txt
```



That is exactly what your assignment expects.

⚠️ One thing to double-check before freezing

Make sure your environment only has what you need.

Sometimes `pip freeze` includes extra junk like:

- unrelated packages
- old dependencies
- large unused libraries

But for this class assignment, this is **totally fine**. No need to over-clean it.

🧠 Quick sanity check for requirements

At minimum, your file should include things like:

```
torch
torchvision
matplotlib
```



```
numpy  
pillow
```

If those are in there, you're good.

🎯 Final verdict

- 👉 Yes — your README is **good enough as-is for submission**
 - 👉 Yes — `pip freeze > requirements.txt` is exactly what you should do
-

🚀 What to do next (final checklist)

You're basically done. Just make sure:

Repo contains:

- ✅ notebook with outputs visible
- ✅ README (this one)
- ✅ requirements.txt
- ✅ outputs/ folder with images
- ✅ A3_Style_Transfer_Report.pdf

Then:

- upload PDF to Canvas
 - paste GitHub link
-

👍 Final note

You went from confusion → full implementation → experiments → documentation

That's exactly what this assignment is testing.

You're in a really good spot now.

📄 👍 💬 ↗️ ↺️ ...

how do i print this conversation as a pdf? I need it to get credit

Good question — and yeah, instructors sometimes ask for this.

I don't have a "print this chat" button from my side, but you can export it yourself pretty easily. Here are the best ways depending on what you're using:

✅ Easiest Method (Works everywhere)

🖨️ Use your browser's print-to-PDF

Step-by-step:

1. Scroll to the **top of this conversation**
2. Make sure all messages are loaded (scroll down fully once)
3. Press:
 - **Mac:** `Cmd + P`
 - **Windows:** `Ctrl + P`

4. In the print menu:
 - Destination → **Save as PDF**
 - Layout → **Portrait**
 - Turn on:
 - Background graphics (if available)
 - Headers/footers OFF (optional cleaner look)
 5. Click **Save**
-

👍 Tips for a clean PDF

Before printing:

- Collapse any side panels (if using ChatGPT web)
 - Zoom out slightly (80–90%) so more content fits per page
 - Scroll through once to ensure everything renders
-

⚠️ If formatting looks weird

Try:

- Change **scale to 90% or 80%**
 - Switch to **Simplified view** (if browser offers it)
 - Use "**More settings → margins → minimum**"
-

🧠 Alternative (if print cuts stuff off)

Copy → Paste → Export

1. Select all chat text (**Cmd+A** / **Ctrl+A**)
2. Copy it
3. Paste into:
 - Google Docs
 - Word
4. Then:
 - File → Download as PDF

This gives you more control (titles, formatting, etc.)

🎯 What your instructor likely expects

Usually they just want:

- proof of interaction / guidance
- not perfect formatting

So the **print-to-PDF method is completely fine**

👍 Recommendation

Use:

👉 **Browser Print → Save as PDF**

It's fastest and accepted in basically all classes.

If you want, I can also help you:

- clean up the conversation into a more "report-style" document
- or extract only the important parts so your PDF isn't huge

